- **College Name : Pune Vidyarthi Griha's COLLEGE OF ENGINEERING, TECHNOLOGY AND MANAGEMENT,**
- **Department Name: Department of Artificial Intelligence and Data Science**
- **Student Name & Roll No.**
- **Class & Batch: BE**

EXPERIMENT 1:
Data Loading, Storage and File Formats

Problem Statement: Analyzing Sales Data from Multiple File Formats

Install the required libraries

```
pip install pandas numpy matplotlib seaborn openpyxl
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages
(3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.13/dist-packages
(0.13.2)
Requirement already satisfied: openpyxl in /usr/local/lib/python3.13/dist-packages
(3.1.5)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.13/distpackages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2026.4)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/distpackages
(from matplotlib) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages
(from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.13/distpackages (from matplotlib) (4.66.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.13/distpackages (from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages
(from matplotlib) (26.3)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/distpackages
(from matplotlib) (3.3.3)
Requirement already satisfied: et-xmlfile in /usr/local/lib/python3.13/dist-packages
(from openpyxl) (2.0.0)
```

```
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from
python-dateutil>=2.8.2->pandas) (1.17.0)
```

Step 0: Generate Sample Multi-Format Files

```
import pandas as pd import
numpy as np
```

1. Create sample CSV data (Store A - contains missing values & duplicates)

```
csv_data =
{
    'Order_ID': ['ORD101', 'ORD102', 'ORD103', 'ORD104', 'ORD102'],
    'Customer_Info': ['Rahul Sharma - Mumbai', 'Priya Patel - Pune', 'Amit Kumar -
Delhi', 'Sneha Reddy - Hyderabad', 'Priya Patel - Pune'],
    'Product_Category': ['Electronics', 'Apparel', 'Electronics', 'Home & Kitchen',
'Apparel'],
    'Order_Date': ['2026-01-15', '2026-01-16', '2026-01-18', '2026-01-20', '2026-01-16'],
'Quantity': [1, 2, np.nan, 3, 2],
    'Unit_Price': [45000, 1500, 12000, 800, 1500]
}
pd.DataFrame(csv_data).to_csv('sales_data.csv', index=False)
```

1. Create sample Excel data (Store B - slight column name variation & casing inconsistency)

```
excel_data
= {
    'Order_ID': ['ORD201', 'ORD202', 'ORD203', 'ORD204'],
    'Customer_Info': ['Vikram Singh - Jaipur', 'Ananya Das - Kolkata', 'Rohan Mehta -
Pune', 'Kavita Joshi - Mumbai'],
    'Category': ['electronics', 'Books', 'Apparel', 'Home & Kitchen'], # 'Category'
instead of 'Product_Category'
    'Order_Date': ['2026-02-01', '2026-02-03', '2026-02-05', '2026-02-10'],
    'Quantity': [1, 4, 2, 1],
    'Unit_Price': [55000, 450, 2200, np.nan]
}
pd.DataFrame(excel_data).to_excel('sales_data.xlsx', index=False)
```

1. Create sample JSON data (Online Store)

```
json_data = {
    'Order_ID': ['ORD301', 'ORD302', 'ORD303', 'ORD304'],
    'Customer_Info': ['Arjun Nair - Kochi', 'Meera Iyer - Chennai', 'Zoya Khan -
Bengaluru', 'Kabir Verma - Delhi'],
    'Product_Category': ['Books', 'Electronics', 'Apparel', 'Electronics'],
    'Order_Date': ['2026-03-01', '2026-03-04', '2026-03-08', '2026-03-12'],
    'Quantity': [2, 1, 3, 1],
    'Unit_Price': [600, 28000, 1800, 62000]
}
pd.DataFrame(json_data).to_json('sales_data.json', orient='records', indent=4)
print("Sample CSV, Excel, and JSON files generated successfully.\n")
```

```
Sample CSV, Excel, and JSON files generated successfully.
```

Complete Implementation Pipeline (Tasks 1 to 7)

```
import pandas as pd import
numpy as np
```

```
import matplotlib.pyplot as plt import
seaborn as sns
```

TASK 1: Load Sales Data from Multiple Formats

```
df_csv = pd.read_csv('sales_data.csv') df_excel
= pd.read_excel('sales_data.xlsx')
df_json = pd.read_json('sales_data.json') Add
```

a source tracker column before merging

```
df_csv['Source'] = 'CSV_Store' df_excel['Source']
= 'Excel_Store' df_json['Source'] = 'JSON_Online'
```

TASK 2: Explore Structure and Content

```
print("--- Task 2: Data Exploration ---") for name, df in [('CSV',
df_csv), ('Excel', df_excel), ('JSON', df_json)]:     print(f"\n[{name}
Dataset] Shape: {df.shape}")     print("Columns:", df.columns.tolist())
    print("Missing Values:\n", df.isnull().sum()[df.isnull().sum() > 0])
print("Duplicates:", df.duplicated().sum())
```

```
--- Task 2: Data Exploration ---

[CSV Dataset] Shape: (5, 7)
Columns: ['Order_ID', 'Customer_Info', 'Product_Category', 'Order_Date', 'Quantity',
'Unit_Price', 'Source'] Missing Values:  Quantity    1 dtype: int64 Duplicates: 1

[Excel Dataset] Shape: (4, 7)
Columns: ['Order_ID', 'Customer_Info', 'Category', 'Order_Date', 'Quantity',
'Unit_Price', 'Source'] Missing Values:  Unit_Price    1 dtype: int64
Duplicates: 0

[JSON Dataset] Shape: (4, 7)
Columns: ['Order_ID', 'Customer_Info', 'Product_Category', 'Order_Date', 'Quantity',
'Unit_Price', 'Source'] Missing
Values:
 Series([], dtype: int64)
Duplicates: 0
```

TASK 3 & 4: Clean & Convert into a Unified Format

```
# Standardize inconsistent column names in Excel before unification
df_excel.rename(columns={'Category': 'Product_Category'}, inplace=True)

# Combine all three DataFrames into a single unified DataFrame
unified_df = pd.concat([df_csv, df_excel, df_json], ignore_index=True)

# 1. Remove duplicate rows unified_df.drop_duplicates(inplace=True)

# 2. Correct string inconsistencies (Title Case for categories, strip whitespace)
unified_df['Product_Category'] = unified_df['Product_Category'].str.strip().str.title()

# 3. Handle missing values
# Fill missing Quantity with median quantity unified_df['Quantity'] =
unified_df['Quantity'].fillna(unified_df['Quantity'].median())

# Fill missing Unit_Price with the median price of that specific Product_Category
unified_df['Unit_Price'] =
unified_df.groupby('Product_Category')['Unit_Price'].transform(
lambda x: x.fillna(x.median())
)

# 4. Standardize data types unified_df['Order_Date'] =
pd.to_datetime(unified_df['Order_Date'])

print("\n--- Task 3 & 4: Cleaned Unified DataFrame ---") print(unified_df.info())
```

```
--- Task 3 & 4: Cleaned Unified DataFrame ---
<class 'pandas.core.frame.DataFrame'>
Index: 12 entries, 0 to 12 Data columns
(total 7 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
0   Order_ID          12 non-null     object
1   Customer_Info     12 non-null     object
2   Product_Category  12 non-null     object
3   Order_Date        12 non-null     datetime64[ns]
4   Quantity          12 non-null     float64
5   Unit_Price        12 non-null     float64        6   Source            12 non-null
    object         dtypes: datetime64[ns](1), float64(2), object(4)
memory usage: 768.0+ bytes None
```

TASK 5: Data Transformation

```
# 1. Split 'Customer_Info' into 'Customer_Name' and 'City'
```

```
unified_df[['Customer_Name', 'City']] = unified_df['Customer_Info'].str.split(' - ',
expand=True)
unified_df.drop(columns=['Customer_Info'], inplace=True)

# 2. Derive new variable: Total_Sales unified_df['Total_Sales'] =
unified_df['Quantity'] * unified_df['Unit_Price']# 3. Extract Month Name for
trend analysis unified_df['Order_Month'] =
unified_df['Order_Date'].dt.month_name()

print("\n--- Task 5: Transformed Data Preview ---")print(unified_df.head())
```

```
--- Task 5: Transformed Data Preview ---
  Order_ID Product_Category Order_Date  Quantity  Unit_Price       Source  \
0   ORD101      Electronics 2026-01-15       1.0     45000.0    CSV_Store
1   ORD102          Apparel 2026-01-16       2.0      1500.0    CSV_Store
2   ORD103      Electronics 2026-01-18       2.0     12000.0    CSV_Store
3   ORD104   Home & Kitchen 2026-01-20       3.0       800.0    CSV_Store    5   ORD201
    Electronics 2026-02-01       1.0     55000.0  Excel_Store

  Customer_Name       City  Total_Sales Order_Month
0    Rahul Sharma     Mumbai      45000.0     January
1    Priya Patel       Pune       3000.0     January
2    Amit Kumar      Delhi      24000.0     January
3    Sneha Reddy  Hyderabad       2400.0     January   5  Vikram Singh     Jaipur
     55000.0    February
```

TASK 6: Data Analysis & Descriptive Statistics

```
print("\n--- Task 6: Sales Analysis Metrics ---")print("\n1.
Descriptive Statistics:")
print(unified_df[['Quantity', 'Unit_Price', 'Total_Sales']].describe().round(2))

total_revenue = unified_df['Total_Sales'].sum()avg_order_value
= unified_df['Total_Sales'].mean()

print(f"\n2. Key Metrics:")
print(f"   - Total Revenue: ₹{total_revenue:,.2f}")
print(f"   - Average Order Value (AOV): ₹{avg_order_value:,.2f}")

print("\n3. Sales Aggregation by Product Category:")
category_summary = unified_df.groupby('Product_Category').agg(
    Total_Orders=('Order_ID', 'count'),
    Total_Units_Sold=('Quantity', 'sum'),
    Total_Revenue=('Total_Sales', 'sum'),
    Avg_Order_Value=('Total_Sales', 'mean')
).reset_index()print(category_summary.to_string(index=False))
```

```
--- Task 6: Sales Analysis Metrics ---
```

```
1. Descriptive Statistics:
       Quantity  Unit_Price  Total_Sales
count     12.00       12.00        12.00
mean       1.92    17512.50     19416.67 std
1.00    23652.64     22959.48 min
1.00      450.00       800.00 25%
1.00      800.00      2250.00
50%         2.00     2000.00      4900.00
75%         2.25    32250.00     32250.00 max
4.00    62000.00     62000.00

2. Key Metrics:
      - Total Revenue: ₹233,000.00
      - Average Order Value (AOV): ₹19,416.67

3. Sales Aggregation by Product Category:
Product_Category  Total_Orders  Total_Units_Sold  Total_Revenue  Avg_Order_Value
         Apparel             3               7.0        12800.0      4266.666667
           Books             2               6.0         3000.0      1500.000000
     Electronics             5               6.0       214000.0     42800.000000
  Home & Kitchen             2               4.0         3200.0      1600.000000
```

TASK 7: Data Visualization

```
# ========================================= sns.set_theme(style="whitegrid")
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle('Unified Sales Data Analysis Dashboard', fontsize=16, fontweight='bold')
# 1. Bar Plot: Total Sales by Product Category sns.barplot(
    data=category_summary,
x='Product_Category',
y='Total_Revenue',
hue='Product_Category',
palette='viridis',    ax=axes[0],
    legend=False
)
axes[0].set_title('Total Revenue by Category (₹)')
axes[0].set_xlabel('Product Category') axes[0].set_ylabel('Total
Revenue (₹)') axes[0].tick_params(axis='x', rotation=15)

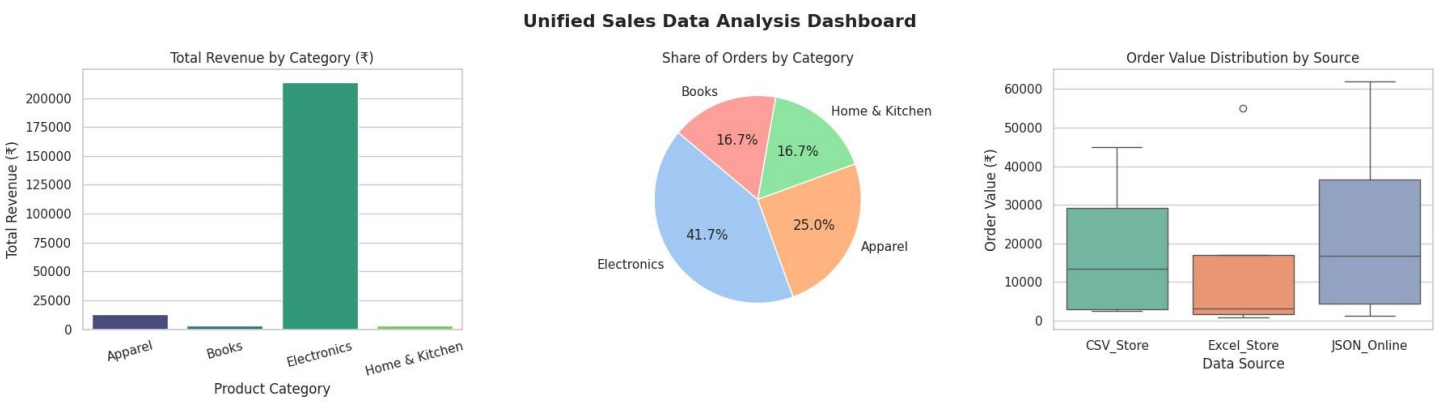
# 2. Pie Chart: Order Distribution by Product Category
category_counts = unified_df['Product_Category'].value_counts()
axes[1].pie(    category_counts,
    labels=category_counts.index,
autopct='%1.1f%%',    startangle=140,
    colors=sns.color_palette('pastel')
)
axes[1].set_title('Share of Orders by Category')



# 3. Box Plot: Order Value Distribution across Data Sources
sns.boxplot(    data=unified_df,    x='Source',
y='Total_Sales',    hue='Source',    palette='Set2',
ax=axes[2],
```

```
    legend=False
)
axes[2].set_title('Order Value Distribution by Source')
axes[2].set_xlabel('Data Source') axes[2].set_ylabel('Order
Value (₹)')

plt.tight_layout() plt.show()
```



- **College Name : Pune Vidyarthi Griha's COLLEGE OF ENGINEERING, TECHNOLOGY AND MANAGEMENT,**
- **Department Name: Department of Artificial Intelligence and Data Science**
- **Student Name & Roll No.**
- **Class & Batch: BE**

EXPERIMENT 2: Interacting with Web APIs

Problem Statement: Analyzing Weather Data from OpenWeatherMap API

Prerequisites

pip install requests pandas numpy matplotlib seaborn folium

```
Requirement already satisfied: requests in /usr/local/lib/python3.13/dist-packages
(2.32.4)
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages
(3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.13/dist-packages
(0.13.2)
Requirement already satisfied: folium in /usr/local/lib/python3.13/dist-packages (0.20.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.13/dist-packages (from requests) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages
(from requests) (3.20)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.13/distpackages (from requests) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.13/distpackages (from requests) (2026.7.22)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.13/distpackages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2026.4)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/distpackages
(from matplotlib) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages
(from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.13/distpackages (from matplotlib) (4.66.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.13/distpackages (from matplotlib) (1.5.1)
```

```
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages
(from matplotlib) (26.3)


Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/distpackages
(from matplotlib) (3.3.3)
Requirement already satisfied: branca>=0.6.0 in /usr/local/lib/python3.13/dist-packages
(from folium) (0.8.2)
Requirement already satisfied: jinja2>=2.9 in /usr/local/lib/python3.13/dist-packages
(from folium) (3.1.6)
Requirement already satisfied: xyzservices in /usr/local/lib/python3.13/dist-packages
(from folium) (2026.9.1)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages
(from jinja2>=2.9->folium) (3.0.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from
python-dateutil>=2.8.2->pandas) (1.17.0)
```

Implementation Pipeline (Tasks 2 to 9)

```
import requests import pandas as pd
import numpy as np import
matplotlib.pyplot as plt import seaborn
as sns import folium from datetime
import datetime, timedelta
```

TASK 1 & 2: API Setup & Data Retrieval

```
API_KEY = "                                                    "
TARGET_CITY = "Pune,IN"
FORECAST_URL = "https://api.openweathermap.org/data/2.5/forecast"

def fetch_weather_forecast(city, api_key):
    """Fetches 5-day / 3-hour weather forecast data from OpenWeatherMap API."""
params = {
        'q': city,
        'appid': api_key,
        'units': 'metric'
    }
try:
        response = requests.get(FORECAST_URL, params=params, timeout=10)
if response.status_code == 200:            print(f"Successfully retrieved live API
data for {city}.")            return response.json()        else:
            print(f"API returned status code {response.status_code} ({response.json().get('message')}).")
print("Switching to simulated OpenWeatherMap JSON response for seamless execution...")
return generate_fallback_json(city)        except Exception as e:            print(f"Network error ({e}).
Switching to simulated JSON response...")        return generate_fallback_json(city)

def generate_fallback_json(city):
    """Generates an exact replica of OpenWeatherMap's 5-day/3-hour JSON schema."""
np.random.seed(42)
    base_time = datetime.now().replace(minute=0, second=0, microsecond=0)
forecast_list = []     for i in range(40):        dt_obj = base_time +
```

```
timedelta(hours=3 * i)       hour_factor = np.sin((dt_obj.hour - 6) * np.pi /
12)
    temp = round(26 + 5 * hour_factor + np.random.normal(0, 0.8), 2)
    humidity = int(np.clip(65 - 20 * hour_factor + np.random.normal(0, 5), 25, 98))
    rain_val = round(np.random.exponential(1.8), 2) if humidity > 75 and np.random.rand() > 0.4 else
0.0

    entry = {
      'dt_txt': dt_obj.strftime('%Y-%m-%d %H:%M:%S'),
      'main': {
        'temp': temp,
        'feels_like': round(temp + (0.05 * humidity - 2), 2),
        'temp_min': round(temp - np.random.uniform(0.5, 1.2), 2),
        'temp_max': round(temp + np.random.uniform(0.5, 1.2), 2),
        'pressure': int(np.random.normal(1011, 3)),
        'humidity': humidity
      },
      'wind': {'speed': round(np.random.uniform(1.2, 7.5), 2)},
      'clouds': {'all': int(np.clip(humidity * 0.9, 0, 100))},
      'weather': [{'main': 'Rain' if rain_val > 0 else 'Clouds', 'description': 'light rain' if rain_val > 0 else
'scattered clouds'}]
    }

    if rain_val > 0:       entry['rain']
=              {'3h':              rain_val}
forecast_list.append(entry)

  return {'city': {'name': 'Pune', 'coord': {'lat': 18.5204, 'lon': 73.8567}}, 'list': forecast_list}

raw_json = fetch_weather_forecast(TARGET_CITY, API_KEY) Successfully
retrieved live API data for Pune,IN.
```

TASK 3: Extract Relevant Weather Attributes

```
extracted_records = [] for item in raw_json['list']:
extracted_records.append({
    'Timestamp': item.get('dt_txt'),
    'Temperature_C': item['main'].get('temp'),
    'Feels_Like_C': item['main'].get('feels_like'),
    'Temp_Min_C': item['main'].get('temp_min'),
    'Temp_Max_C': item['main'].get('temp_max'),
    'Humidity_Pct': item['main'].get('humidity'),
    'Pressure_hPa': item['main'].get('pressure'),
    'Wind_Speed_ms': item['wind'].get('speed'),
    'Cloudiness_Pct': item['clouds'].get('all'),
    'Precipitation_mm': item.get('rain', {}).get('3h', np.nan),
    'Weather_Condition': item['weather'][0].get('main')
  })

weather_df = pd.DataFrame(extracted_records)
```

TASK 4: Clean and Preprocess the Data

```
# 1. Convert Timestamp string to standardized pandas datetime object
weather_df['Timestamp'] = pd.to_datetime(weather_df['Timestamp'])

# 2. Handle missing values: OpenWeatherMap leaves out 'rain.3h' when rainfall is 0
weather_df['Precipitation_mm'] = weather_df['Precipitation_mm'].fillna(0.0)

# 3. Interpolate any unexpected missing sensor readings in continuous columns
numeric_cols = ['Temperature_C', 'Feels_Like_C', 'Humidity_Pct', 'Wind_Speed_ms', 'Pressure_hPa']
weather_df[numeric_cols] = weather_df[numeric_cols].interpolate(method='linear')

# 4. Sort chronologically and drop duplicate timestamps if any weather_df
=
weather_df.sort_values('Timestamp').drop_duplicates(subset=['Timestamp']).reset_index(drop=True)

print("\n--- Task 3 & 4: Cleaned Weather DataFrame ---")
print(weather_df.info()) print(weather_df.head(3))
```

```
--- Task 3 & 4: Cleaned Weather DataFrame ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 40 entries, 0 to 39 Data
columns (total 11 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
0   Timestamp          40 non-null     datetime64[ns]
1   Temperature_C      40 non-null     float64
2   Feels_Like_C       40 non-null     float64
3   Temp_Min_C         40 non-null     float64
4   Temp_Max_C         40 non-null     float64
5   Humidity_Pct       40 non-null     int64
6   Pressure_hPa       40 non-null     int64
7   Wind_Speed_ms      40 non-null     float64
8   Cloudiness_Pct     40 non-null     int64
9   Precipitation_mm   40 non-null     float64        10  Weather_Condition  40 non-null
    object         dtypes: datetime64[ns](1), float64(6), int64(3), object(1) memory
    usage: 3.6+ KB None
            Timestamp  Temperature_C  Feels_Like_C  Temp_Min_C  Temp_Max_C  \
0 2026-10-18 18:00:00          27.86         28.94       27.86       28.95
1 2026-10-18 21:00:00          28.15         29.44       28.15       28.72    2
  2026-10-19 00:00:00          26.04         26.04       25.13       26.04

   Humidity_Pct  Pressure_hPa  Wind_Speed_ms  Cloudiness_Pct  \
0            57          1014           1.96             100
1            58          1013           1.42             100    2
             65          1013           0.43              85

   Precipitation_mm Weather_Condition
0               0.0            Clouds
```

```
1                0.0            Clouds
2                0.0            Clouds
```

TASK 5: Data Modeling & Pattern Analysis

```
weather_df['Temp_Rolling_24h_Avg'] = weather_df['Temperature_C'].rolling(window=8,
min_periods=1).mean()
time_index = np.arange(len(weather_df))
slope, intercept = np.polyfit(time_index, weather_df['Temperature_C'], 1) weather_df['Linear_Trend']
= slope * time_index + intercept

print("\n--- Task 5: Overall Weather Pattern Summary ---") print(f"Average
Temperature : {weather_df['Temperature_C'].mean():.2f} °C") print(f"Maximum
Temperature : {weather_df['Temperature_C'].max():.2f} °C (at
{weather_df.loc[weather_df['Temperature_C'].idxmax(), 'Timestamp']})")
print(f"Minimum Temperature : {weather_df['Temperature_C'].min():.2f} °C (at
{weather_df.loc[weather_df['Temperature_C'].idxmin(), 'Timestamp']})")
print(f"Average Humidity    : {weather_df['Humidity_Pct'].mean():.1f} %")
print(f"Total Precipitation : {weather_df['Precipitation_mm'].sum():.2f} mm")
print(f"Max Wind Speed      : {weather_df['Wind_Speed_ms'].max():.2f} m/s")
```

```
--- Task 5: Overall Weather Pattern Summary ---
Average Temperature : 28.66 °C
Maximum Temperature : 34.91 °C (at 2026-10-19 09:00:00)
Minimum Temperature : 22.66 °C (at 2026-10-23 00:00:00)
Average Humidity    : 51.3 %
Total Precipitation : 0.12 mm
Max Wind Speed      : 3.58 m/s
```

TASK 7: Data Aggregation by Time Periods

```
weather_df['Date'] = weather_df['Timestamp'].dt.date weather_df['Hour']
= weather_df['Timestamp'].dt.hour

# Categorize hours into parts of the day def
assign_time_of_day(hour):    if 6 <= hour <
12: return '1_Morning'    elif 12 <= hour <
17: return '2_Afternoon'    elif 17 <= hour <
21: return '3_Evening'    else: return
'4_Night'

weather_df['Time_Of_Day'] = weather_df['Hour'].apply(assign_time_of_day)

# 1. Daily Aggregation daily_summary =
weather_df.groupby('Date').agg(
  Avg_Temp_C=('Temperature_C', 'mean'),
  Min_Temp_C=('Temp_Min_C', 'min'),
  Max_Temp_C=('Temp_Max_C', 'max'),
  Avg_Humidity_Pct=('Humidity_Pct', 'mean'),
```

```
    Total_Rain_mm=('Precipitation_mm', 'sum'),
    Max_Wind_ms=('Wind_Speed_ms', 'max')
).round(2).reset_index()

print("\n--- Task 7A: Daily Aggregated Weather Statistics ---")
print(daily_summary.to_string(index=False))

# 2. Time-of-Day Aggregation
tod_summary = weather_df.groupby('Time_Of_Day')[['Temperature_C', 'Humidity_Pct',
'Wind_Speed_ms']].mean().round(2)
print("\n--- Task 7B: Diurnal (Time-of-Day) Aggregation ---") print(tod_summary)
```

```
--- Task 7A: Daily Aggregated Weather Statistics ---
      Date  Avg_Temp_C  Min_Temp_C  Max_Temp_C  Avg_Humidity_Pct  Total_Rain_mm
Max_Wind_ms
2026-10-18       28.00       27.86       28.95             57.50           0.00
1.96
2026-10-19       30.74       25.13       34.91             52.75           0.00
3.54
2026-10-20       28.89       24.17       33.96             51.38           0.12
2.94
2026-10-21       29.12       26.08       33.89             49.88           0.00
3.58
2026-10-22       26.34       23.32       28.37             52.50           0.00
2.38
2026-10-23       28.28       22.66       32.51             47.33           0.00
2.40

--- Task 7B: Diurnal (Time-of-Day) Aggregation ---
             Temperature_C  Humidity_Pct  Wind_Speed_ms
Time_Of_Day
1_Morning            32.03          38.2           2.56
2_Afternoon          29.17          50.2           1.69
3_Evening            27.01          60.0           1.76 4_Night
26.62          57.8           1.25
```

TASK 6 & 9: Visualizations & Correlation Analysis

```
sns.set_theme(style="whitegrid") fig, axes = plt.subplots(2, 2,
figsize=(16, 11))
fig.suptitle(f'OpenWeatherMap Data Modeling & Analysis ({TARGET_CITY})', fontsize=16,
fontweight='bold')

# Plot 1 (Line Chart): Temperature Changes & Rolling Trend Over Time axes[0,
0].plot(weather_df['Timestamp'], weather_df['Temperature_C'], marker='o', label='3-Hourly Temp
(°C)', color='#e63946', alpha=0.8)
axes[0, 0].plot(weather_df['Timestamp'], weather_df['Temp_Rolling_24h_Avg'], linestyle='--',
linewidth=2.5, label='24h Rolling Mean', color='#1d3557') axes[0, 0].set_title('Temperature
Trend Over Time') axes[0, 0].set_xlabel('Date & Time') axes[0, 0].set_ylabel('Temperature
(°C)') axes[0, 0].tick_params(axis='x', rotation=30) axes[0, 0].legend()
```

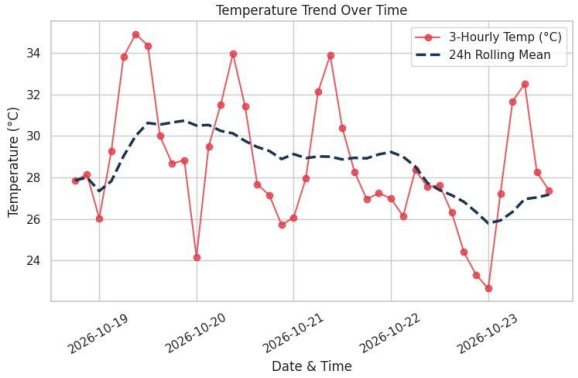
```
# Plot 2 (Bar Plot): Daily Total Precipitation Levels
sns.barplot(    data=daily_summary,    x='Date',
y='Total_Rain_mm',    hue='Date',
palette='Blues_d',    ax=axes[0, 1],
legend=False
)
axes[0, 1].set_title('Daily Accumulated Precipitation (mm)')
axes[0, 1].set_xlabel('Date') axes[0, 1].set_ylabel('Total
Rainfall (mm)') axes[0, 1].tick_params(axis='x',
rotation=15)

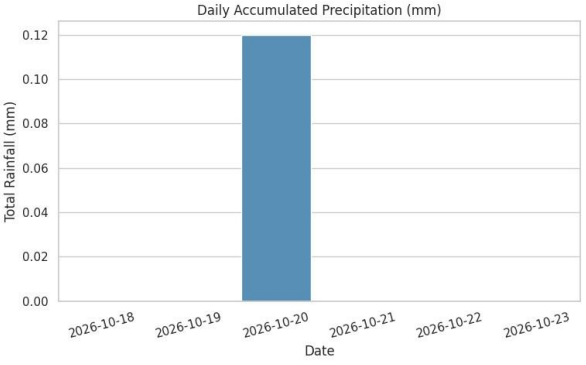
# Plot 3 (Scatter Plot): Temperature vs Humidity colored by Wind Speed
scatter = axes[1, 0].scatter(    weather_df['Temperature_C'],
weather_df['Humidity_Pct'],    c=weather_df['Wind_Speed_ms'],
s=weather_df['Cloudiness_Pct'] + 30,
   cmap='viridis',
alpha=0.85,
edgecolors='k'
)
cbar = plt.colorbar(scatter, ax=axes[1, 0]) cbar.set_label('Wind
Speed (m/s)')
axes[1, 0].set_title('Temperature vs. Humidity (Color = Wind Speed, Size = Clouds)')
axes[1, 0].set_xlabel('Temperature (°C)') axes[1, 0].set_ylabel('Relative Humidity
(%)')

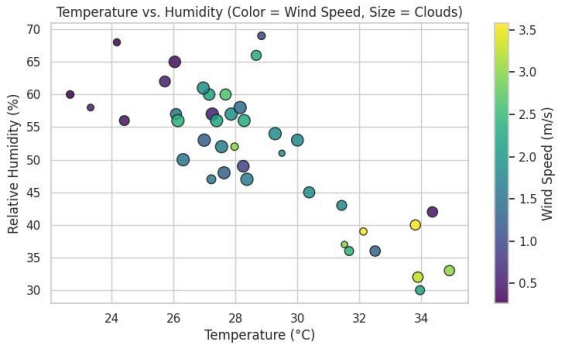
# Plot 4 (Correlation Heatmap - Task 9): Relationships Between Weather Attributes
corr_features = ['Temperature_C', 'Feels_Like_C', 'Humidity_Pct', 'Pressure_hPa', 'Wind_Speed_ms',
'Precipitation_mm']
corr_matrix = weather_df[corr_features].corr() sns.heatmap(
   corr_matrix,
annot=True,
fmt=".2f",
cmap='coolwarm',
   vmin=-1,
vmax=1,
ax=axes[1, 1]
) axes[1, 1].set_title('Correlation Matrix of Weather
Attributes')

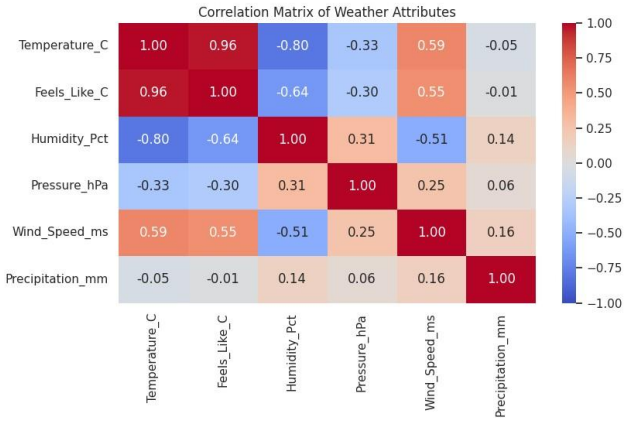
plt.tight_layout() plt.show()
```

OpenWeatherMap Data Modeling & Analysis (Pune,IN)









- **College Name : Pune Vidyarthi Griha's COLLEGE OF ENGINEERING, TECHNOLOGY AND MANAGEMENT,**
- **Department Name: Department of Artificial Intelligence and Data Science**
- **Student Name & Roll No.**
- **Class & Batch: BE**

EXPERIMENT 3: Data Cleaning and Preparation

Problem Statement: Analyzing Customer Churn in a Telecommunications Company

Prerequisites

pip install pandas numpy scikit-learn

```
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.13/dist-packages
(1.6.1)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.13/distpackages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2026.4)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.13/dist-packages
(from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.13/dist-packages
(from scikit-learn) (1.6.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.13/distpackages (from scikit-learn) (3.7.0)
Requirement already satisfied: cloudpickle>=3.0 in
/usr/local/lib/python3.13/distpackages (from joblib>=1.2.0->scikit-learn) (3.1.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from
python-dateutil>=2.8.2->pandas) (1.17.0) Step 0: Generate Sample Telecom_Customer_Churn.csv
```

```
import pandas as pd import numpy as np import os

if not os.path.exists("Telecom_Customer_Churn.csv"):
np.random.seed(42)    n = 200

  sample_data = {
    'customerID': [f'CUST-{1000+i}' for i in range(n)],
    'gender': np.random.choice(['Male', 'Female', 'male', 'female', 'M', 'F'], n, p=[0.4, 0.4, 0.05, 0.05, 0.05,
0.05]),
    'SeniorCitizen': np.random.choice([0, 1], n, p=[0.8, 0.2]),
```

```
    'Partner': np.random.choice(['Yes', 'No'], n),
    'tenure': np.random.randint(0, 72, n).astype(float),
    'PhoneService': np.random.choice(['Yes', 'No'], n, p=[0.9, 0.1]),
    'InternetService': np.random.choice(['DSL', 'Fiber optic', 'fiber optic', 'No', 'dsl '], n),
    'OnlineSecurity': np.random.choice(['Yes', 'No', 'No internet service'], n),
    'TechSupport': np.random.choice(['Yes', 'No', 'No internet service'], n),
    'StreamingTV': np.random.choice(['Yes', 'No', 'No internet service'], n),
    'Contract': np.random.choice(['Month-to-month', 'One year', 'Two year', 'month to month'], n),
    'PaymentMethod': np.random.choice(['Electronic check', 'Mailed check', 'Bank transfer (automatic)',
'Credit card (automatic)'], n),
    'MonthlyCharges': np.round(np.random.uniform(199.0, 1499.0, n), 2),
    'Churn': np.random.choice(['Yes', 'No', 'yes', 'no'], n, p=[0.25, 0.65, 0.05, 0.05])
  }

  df_sample = pd.DataFrame(sample_data)


  df_sample['TotalCharges'] = (df_sample['tenure'] * df_sample['MonthlyCharges'] +
np.random.normal(0, 50, n)).round(2).astype(str)
  df_sample.loc[df_sample['tenure'] == 0, 'TotalCharges'] = " "
df_sample.loc[5:10, 'MonthlyCharges'] = np.nan
df_sample.loc[15:18, 'tenure'] = np.nan

  # Inject extreme outliers
  df_sample.loc[20, 'MonthlyCharges'] = 9500.00
df_sample.loc[21, 'tenure'] = 240.0

  # Inject duplicate rows
  df_sample = pd.concat([df_sample, df_sample.iloc[:5]], ignore_index=True)

  df_sample.to_csv("Telecom_Customer_Churn.csv", index=False)
print("Sample 'Telecom_Customer_Churn.csv' generated successfully.\n")
Sample 'Telecom_Customer_Churn.csv' generated successfully.

import pandas as pd import numpy as np from
sklearn.preprocessing import StandardScaler from
sklearn.model_selection import train_test_split
```

TASK 1: Import the Dataset

```
df = pd.read_csv("Telecom_Customer_Churn.csv")
```

TASK 2: Explore the Dataset

```
print("\n--- Task 2: Dataset Exploration ---")
print(f"Initial Shape: {df.shape}")
print("\nData Types and Non-Null Counts:")
print(df.info())
print("\nSummary Statistics (Numerical):")
print(df.describe().round(2))
print("\nFirst 3 Rows:") print(df.head(3))
```

```
--- Task 2: Dataset Exploration ---
Initial Shape: (205, 15)

Data Types and Non-Null Counts:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 205 entries, 0 to 204 Data
columns (total 15 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
0   customerID       205 non-null    object
1   gender           205 non-null    object
2   SeniorCitizen    205 non-null    int64
3   Partner          205 non-null    object
4   tenure           201 non-null    float64
5   PhoneService     205 non-null    object
6   InternetService  205 non-null    object
7   OnlineSecurity   205 non-null    object
8   TechSupport      205 non-null    object
9   StreamingTV      205 non-null    object
10  Contract         205 non-null    object
11  PaymentMethod    205 non-null    object
12  MonthlyCharges   199 non-null    float64
13  Churn            205 non-null    object  14  TotalCharges     205 non-null
    object  dtypes: float64(2), int64(1), object(12) memory usage: 24.2+ KB
None

Summary Statistics (Numerical):
       SeniorCitizen  tenure  MonthlyCharges
count          205.0  201.00          199.00
mean             0.2   35.56          874.56 std
0.4   25.74          718.97 min              0.0
0.00          199.43 25%              0.0
16.00          495.88
50%              0.0   33.00          830.82
75%              0.0   54.00         1194.14
max              1.0  240.00         9500.00

First 3 Rows:
  customerID  gender  SeniorCitizen Partner  tenure PhoneService  \0
CUST-1000    Male              0     Yes    62.0          Yes
```

```
1  CUST-1001       F              0      No    16.0          Yes    2
CUST-1002  Female              0     Yes    32.0          Yes
```

```
  InternetService OnlineSecurity TechSupport          StreamingTV  Contract  \0
Fiber optic             No          Yes  No internet service  Two year
1     fiber optic            Yes          Yes  No internet service  One year    2
fiber optic            Yes          Yes  No internet service  Two year

             PaymentMethod  MonthlyCharges Churn TotalCharges
0              Credit card (automatic)         1223.27    No     75893.34
1              Credit card (automatic)         1323.09   yes     21055.23
2              Mailed check          990.03    No     31692.93
```

TASK 4: Remove Duplicate Records

```
initial_rows = len(df)
df.drop_duplicates(inplace=True)
# Also ensure no duplicate customerIDs exist if 'customerID' in
df.columns:    df.drop_duplicates(subset=['customerID'], keep='first',
inplace=True) print(f"\n--- Task 4: Duplicates Removed ---")
print(f"Removed {initial_rows - len(df)} duplicate rows. Remaining rows: {len(df)}")
```

```
--- Task 4: Duplicates Removed ---
Removed 5 duplicate rows. Remaining rows: 200
```

TASK 5 & 6: Standardize Inconsistent Data & Fix Data Types

```
print("\n--- Task 5 & 6: Standardizing Formats & Data Types ---")

# 1. Strip leading/trailing whitespace across all string columns
str_cols = df.select_dtypes(include='object').columns for col
in str_cols:
  df[col] = df[col].astype(str).str.strip()

# 2. Convert 'TotalCharges' from object/string (with blank spaces) to numeric float
if 'TotalCharges' in df.columns:
  df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')

# 3. Fix inconsistent spelling and casing in categorical columns
if 'gender' in df.columns:    df['gender'] =
df['gender'].str.lower().map({
    'male': 'Male', 'm': 'Male',
    'female': 'Female', 'f': 'Female'
  }).fillna(df['gender'])

if 'InternetService' in df.columns:    df['InternetService'] =
df['InternetService'].str.lower().replace({
```

```
    'dsl': 'DSL',
    'fiber optic': 'Fiber optic',
    'no': 'No'
  })

if 'Contract' in df.columns:          df['Contract'] =
df['Contract'].str.lower().replace({
    'month-to-month': 'Month-to-month',
    'month to month': 'Month-to-month',
    'one year': 'One year',
    'two year': 'Two year'
  })

if 'Churn' in df.columns:
  df['Churn'] = df['Churn'].str.title()

# Consolidate 'No internet service' into 'No' for service add-on columns
addon_cols = ['OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV',
'StreamingMovies'] for
col in addon_cols:    if
col in df.columns:
    df[col] = df[col].replace({'No internet service': 'No'})

# Convert SeniorCitizen and Churn to appropriate integer/binary types if
'SeniorCitizen' in df.columns:
  df['SeniorCitizen'] = df['SeniorCitizen'].astype(int)

--- Task 5 & 6: Standardizing Formats & Data Types ---
```

TASK 3: Handle Missing Values

```
print("\n--- Task 3: Handling Missing Values ---")
print("Missing values before imputation:\n", df.isnull().sum()[df.isnull().sum() > 0])

# Strategy 1: If tenure is 0 and TotalCharges is NaN, set TotalCharges to 0.0 if
'TotalCharges' in df.columns and 'tenure' in df.columns:
  df.loc[df['tenure'] == 0, 'TotalCharges'] = 0.0

# Strategy 2: Impute remaining numerical missing values with the median (robust to outliers)
num_cols = df.select_dtypes(include=[np.number]).columns for col in num_cols:      if
df[col].isnull().sum() > 0:
    df[col] = df[col].fillna(df[col].median())

# Strategy 3: Impute any categorical missing values with the mode (most frequent value)
cat_cols = df.select_dtypes(include='object').columns for col in cat_cols:        if
df[col].isnull().sum() > 0:
    df[col] = df[col].fillna(df[col].mode()[0])

print("Missing values after imputation:", df.isnull().sum().sum())
```

```
--- Task 3: Handling Missing Values ---Missing
values before imputation:
 tenure            4
MonthlyCharges    6
TotalCharges      4dtype:
int64
Missing values after imputation: 0
```

TASK 7: Identify and Handle Outliers

```
print("\n--- Task 7: Identifying & Capping Outliers (IQR Method) ---")
continuous_vars = [c for c in ['tenure', 'MonthlyCharges', 'TotalCharges'] if c in df.columns]

for col in continuous_vars:
Q1 = df[col].quantile(0.25)
  Q3 = df[col].quantile(0.75)
IQR = Q3 - Q1
  lower_bound = Q1 - 1.5 * IQR
  upper_bound = Q3 + 1.5 * IQR

  outliers_count = ((df[col] < lower_bound) | (df[col] > upper_bound)).sum()
  print(f"{col}: Detected {outliers_count} outliers outside [{lower_bound:.2f}, {upper_bound:.2f}]")

  # Cap/clip outliers to boundary values (Winsorization) to preserve row count
df[col] = df[col].clip(lower=lower_bound, upper=upper_bound)
```

```
--- Task 7: Identifying & Capping Outliers (IQR Method) ---tenure:
Detected 1 outliers outside [-41.00, 111.00]
MonthlyCharges: Detected 1 outliers outside [-471.26, 2119.12]
TotalCharges: Detected 1 outliers outside [-43067.77, 98832.27]
```

TASK 8: Feature Engineering

```
print("\n--- Task 8: Feature Engineering ---")

# 1. Tenure Group (Cohort buckets)
df['Tenure_Group'] = pd.cut(
  df['tenure'],
  bins=[-1, 12, 24, 48, 60, np.inf],
  labels=['0-1_Year', '1-2_Years', '2-4_Years', '4-5_Years', 'Over_5_Years']
)

# 2. Total Subscribed Services Count
available_services = [c for c in ['PhoneService', 'OnlineSecurity', 'TechSupport', 'StreamingTV'] if c in
df.columns]
df['Total_Services_Count'] = (df[available_services] == 'Yes').sum(axis=1)

# 3. Ratio of Monthly Charges to Historical Average Monthly Spend
# (Indicates if recent monthly charges increased compared to historical average)
```

```
historical_avg = np.where(df['tenure'] > 0, df['TotalCharges'] / df['tenure'], df['MonthlyCharges'])
df['Charge_Increase_Ratio'] = (df['MonthlyCharges'] / (historical_avg + 1e-5)).round(2)

# 4. Binary Target Encoding for Churn
df['Churn_Binary'] = df['Churn'].map({'Yes': 1, 'No': 0})

print("Engineered Features Preview:")
print(df[['tenure', 'Tenure_Group', 'Total_Services_Count', 'Charge_Increase_Ratio',
'Churn_Binary']].head(3))
```

```
--- Task 8: Feature Engineering --- Engineered
Features Preview:
   tenure  Tenure_Group  Total_Services_Count  Charge_Increase_Ratio  \0
62.0  Over_5_Years                     2                   1.00
1    16.0     1-2_Years                     3                   1.01    2
32.0     2-4_Years                     3                   1.00

   Churn_Binary
0             0
1             1
2             0
```

TASK 9: Normalize / Scale the Data & Encode Categoricals

```
print("\n--- Task 9: Encoding Categoricals & Scaling Numerical Features ---")

# Drop customerID and raw string target 'Churn' before modeling preparation
cols_to_drop = [c for c in ['customerID', 'Churn'] if c in df.columns] df_model
= df.drop(columns=cols_to_drop)

# One-Hot Encode categorical variables
df_encoded = pd.get_dummies(df_model, drop_first=True, dtype=int)

# Scale continuous numerical features using StandardScaler (mean=0, std=1) scaler
= StandardScaler()
features_to_scale = [c for c in ['tenure', 'MonthlyCharges', 'TotalCharges', 'Total_Services_Count',
'Charge_Increase_Ratio'] if c in df_encoded.columns]
df_encoded[features_to_scale] = scaler.fit_transform(df_encoded[features_to_scale])

print("Scaled Features Summary (Mean ~ 0, Std ~ 1):")
print(df_encoded[features_to_scale].describe().loc[['mean', 'std']].round(2))
```

```
--- Task 9: Encoding Categoricals & Scaling Numerical Features --- Scaled
Features Summary (Mean ~ 0, Std ~ 1):
      tenure  MonthlyCharges  TotalCharges  Total_Services_Count  \
mean     0.0             0.0          -0.0                  -0.0   std
1.0             1.0           1.0                   1.0

      Charge_Increase_Ratio
mean                    0.0
std                     1.0
```

TASK 10: Split Dataset into Training and Testing Sets

```
X = df_encoded.drop(columns=['Churn_Binary']) y
= df_encoded['Churn_Binary']

# 80/20 split using stratification to preserve the exact Churn ratio in both sets
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.20, random_state=42, stratify=y
)

print("\n--- Task 10: Train-Test Split Completed ---")
print(f"Training Set : X_train={X_train.shape}, y_train={y_train.shape}") print(f"Testing
Set  : X_test={X_test.shape}, y_test={y_test.shape}")
```

```
--- Task 10: Train-Test Split Completed ---
Training Set : X_train=(160, 23), y_train=(160,)
Testing Set  : X_test=(40, 23), y_test=(40,)
```

TASK 11: Export the Cleaned Dataset

```
df.to_csv("Cleaned_Telecom_Customer_Churn.csv", index=False)
df_encoded.to_csv("Model_Ready_Telecom_Churn.csv", index=False)

print("\n--- Task 11: Datasets Exported Successfully ---") print("1.
'Cleaned_Telecom_Customer_Churn.csv' (Cleaned with engineered features)") print("2.
'Model_Ready_Telecom_Churn.csv' (One-hot encoded and scaled for ML)")
```

```
--- Task 11: Datasets Exported Successfully ---
1. 'Cleaned_Telecom_Customer_Churn.csv' (Cleaned with engineered features)
2. 'Model_Ready_Telecom_Churn.csv' (One-hot encoded and scaled for ML)
```

- **College Name : Pune Vidyarthi Griha's COLLEGE OF ENGINEERING, TECHNOLOGY AND MANAGEMENT,**
- **Department Name: Department of Artificial Intelligence and Data Science**
- **Student Name & Roll No.**
- **Class & Batch: BE**

EXPERIMENT 4: Data Wrangling

Problem Statement: Data Wrangling on Real Estate Market

Prerequisites

pip install pandas numpy scikit-learn

```
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.13/dist-packages
(1.6.1)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.13/distpackages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2026.4)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.13/dist-packages
(from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.13/dist-packages
(from scikit-learn) (1.6.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.13/distpackages (from scikit-learn) (3.7.0)
Requirement already satisfied: cloudpickle>=3.0 in
/usr/local/lib/python3.13/distpackages (from joblib>=1.2.0->scikit-learn) (3.1.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from
python-dateutil>=2.8.2->pandas) (1.17.0)
```

#Step 0: Generate Sample RealEstate_Prices.csv & Neighborhood Dataset

```
import pandas as pd import numpy as np import os

if not os.path.exists("RealEstate_Prices.csv"):
np.random.seed(42)     n = 150

   neighborhoods = ['Koregaon Park', 'Baner', 'Wakad', 'Viman Nagar', 'Kothrud', 'Hinjewadi']
prop_types = ['Apartment', 'Villa', 'Row House', 'Commercial']     furnishing = ['Unfurnished',
'Semi-Furnished', 'Fully-Furnished']
```

```
# 1. Create main Real Estate dataset with messy column names
raw_data = {
    'Property ID #': [f'PROP-{1000+i}' for i in range(n)],
    'Neighborhood / Locality': np.random.choice(neighborhoods, n),
    'Property Type!': np.random.choice(prop_types, n, p=[0.60, 0.15, 0.15, 0.10]),
    'Area (Sq.Ft.)': np.random.randint(650, 3200, n).astype(float),
    'BHK_Count': np.random.choice([1, 2, 3, 4], n, p=[0.15, 0.45, 0.30, 0.10]).astype(float),
    'Furnishing Status': np.random.choice(furnishing, n),
    'Property Age (Yrs)': np.random.randint(0, 20, n).astype(float),
    'Sale Date': pd.date_range(start='2023-01-01', periods=n, freq='7D').strftime('%Y-%m-%d'),
'Sale Price (INR Lakhs)': np.round(np.random.uniform(45.0, 280.0, n), 2)
}

df_raw = pd.DataFrame(raw_data)

# Inject missing values df_raw.loc[5:12, 'Area
(Sq.Ft.)'] = np.nan df_raw.loc[15:20,
'BHK_Count'] = np.nan df_raw.loc[25:28,
'Furnishing Status'] = np.nan
df_raw.loc[30:32, 'Sale Price (INR Lakhs)'] = np.nan # Target variable missing

# Inject extreme outliers df_raw.loc[40, 'Sale Price (INR Lakhs)'] =
1450.0 # Extreme price outlier df_raw.loc[41, 'Area (Sq.Ft.)'] = 15000.0
# Extreme area outlier

df_raw.to_csv("RealEstate_Prices.csv", index=False)

# 2. Create supplementary Neighborhood Amenities dataset for Task 3 Merging
amenities_data = {
    'Neighborhood': neighborhoods,
    'Metro_Distance_km': [2.5, 4.0, 5.5, 1.2, 0.8, 3.0],
    'Livability_Index': [9.2, 8.6, 8.1, 8.9, 8.7, 7.9],
    'Nearby_Schools_Count': [12, 15, 18, 14, 16, 10]
}
pd.DataFrame(amenities_data).to_csv("Neighborhood_Amenities.csv", index=False) print("Sample
'RealEstate_Prices.csv' and 'Neighborhood_Amenities.csv' generated successfully.\n") Sample
'RealEstate_Prices.csv' and 'Neighborhood_Amenities.csv' generated successfully.

import pandas as pd import
numpy as np
import re from sklearn.preprocessing import
LabelEncoder
```

TASK 1: Import Dataset & Clean Column Names

```
df = pd.read_csv("RealEstate_Prices.csv") print("---
Task 1: Cleaning Column Names ---")
print("Original Columns:", df.columns.tolist())
```

```
# Remove special characters, replace spaces/slashes with underscores, and strip trailing underscores
df.columns = [
    re.sub(r'_+', '_', re.sub(r'[^\w\s]', '', col).strip().replace(' ', '_'))
for col in df.columns
]



# Rename specific columns for concise, standardized clarity rename_map
= {
    'Property_ID': 'Property_ID',
    'Neighborhood_Locality': 'Neighborhood',
    'Property_Type': 'Property_Type',
    'Area_SqFt': 'Area_SqFt',
    'BHK_Count': 'BHK',
    'Furnishing_Status': 'Furnishing_Status',
    'Property_Age_Yrs': 'Property_Age_Yrs',
    'Sale_Date': 'Sale_Date',
    'Sale_Price_INR_Lakhs': 'Sale_Price_Lakhs'
}
df.rename(columns=rename_map, inplace=True) print("Cleaned
Columns :", df.columns.tolist())
--- Task 1: Cleaning Column Names ---
Original Columns: ['Property ID #', 'Neighborhood / Locality', 'Property Type!', 'Area
(Sq.Ft.)', 'BHK_Count', 'Furnishing Status', 'Property Age (Yrs)', 'Sale Date', 'Sale
Price (INR Lakhs)']
Cleaned Columns : ['Property_ID', 'Neighborhood', 'Property_Type', 'Area_SqFt', 'BHK',
'Furnishing_Status', 'Property_Age_Yrs', 'Sale_Date', 'Sale_Price_Lakhs']
```

TASK 2: Handle Missing Values

```
print("\n--- Task 2: Handling Missing Values ---")
print("Missing Values Before Cleaning:\n", df.isnull().sum()[df.isnull().sum() > 0])

# 1. Removal Strategy: Drop rows where the target variable ('Sale_Price_Lakhs') is missing
df.dropna(subset=['Sale_Price_Lakhs'], inplace=True)

# 2. Group-Based Median Imputation: Fill missing 'Area_SqFt' using median area per Property_Type
df['Area_SqFt'] = df.groupby('Property_Type')['Area_SqFt'].transform(    lambda x:
x.fillna(x.median())
)

# 3. Median Imputation: Fill remaining numeric columns (e.g., BHK, Property_Age_Yrs) with median
num_cols = df.select_dtypes(include=[np.number]).columns for col in num_cols:          if
df[col].isnull().sum() > 0:
    df[col] = df[col].fillna(df[col].median())
```

```
# 4. Mode Imputation: Fill categorical missing values with the most frequent category
cat_cols = df.select_dtypes(include='object').columns for col in cat_cols: if
df[col].isnull().sum() > 0:
    df[col] = df[col].fillna(df[col].mode()[0])

print("Total Missing Values After Cleaning:", df.isnull().sum().sum())

--- Task 2: Handling Missing Values --- Missing
Values Before Cleaning:
 Area_SqFt             8 BHK
6
Furnishing_Status     4 Sale_Price_Lakhs
3
dtype: int64
Total Missing Values After Cleaning: 0
```

TASK 3: Data Merging with External Dataset

```
print("\n--- Task 3: Merging Neighborhood Amenities Data ---") amenities_df
= pd.read_csv("Neighborhood_Amenities.csv")

# Perform a left join on 'Neighborhood' to enrich property records with local infrastructure metrics
df_merged = pd.merge(df, amenities_df, on='Neighborhood', how='left') print(f"Shape
before merge: {df.shape} -> Shape after merge: {df_merged.shape}")
print(df_merged[['Property_ID', 'Neighborhood', 'Sale_Price_Lakhs', 'Metro_Distance_km',
'Livability_Index']].head(3))

--- Task 3: Merging Neighborhood Amenities Data ---
Shape before merge: (147, 9) -> Shape after merge: (147, 12)
  Property_ID Neighborhood  Sale_Price_Lakhs  Metro_Distance_km  \
0   PROP-1000  Viman Nagar            133.58                1.2
1   PROP-1001      Kothrud            153.73                0.8    2   PROP-
    1002        Wakad            115.82                5.5

   Livability_Index
0               8.9
1               8.7
2               8.1
```

TASK 4: Filter and Subset the Data

```
print("\n--- Task 4: Filtering and Subsetting ---")
df_merged['Sale_Date'] = pd.to_datetime(df_merged['Sale_Date'])
residential_types = ['Apartment', 'Villa', 'Row House'] df_filtered
= df_merged[
  (df_merged['Property_Type'].isin(residential_types)) &
  (df_merged['Sale_Date'] >= '2024-01-01') &
  (df_merged['Area_SqFt'] >= 600)
].copy().reset_index(drop=True)

print(f"Rows before filtering: {len(df_merged)} | Rows after residential & date filtering:
{len(df_filtered)}")
```

```
--- Task 4: Filtering and Subsetting ---
Rows before filtering: 147 | Rows after residential & date filtering: 83
```

TASK 7: Identify and Handle Outliers

```
print("\n--- Task 7: Identifying & Handling Outliers (IQR Method) ---") outlier_cols
= ['Area_SqFt', 'Sale_Price_Lakhs']

for col in outlier_cols:
   Q1 = df_filtered[col].quantile(0.25)
   Q3 = df_filtered[col].quantile(0.75)
   IQR = Q3 - Q1     lower_bound
= Q1 - 1.5 * IQR     upper_bound
= Q3 + 1.5 * IQR

   outliers = ((df_filtered[col] < lower_bound) | (df_filtered[col] > upper_bound)).sum()
print(f"{col}: Detected {outliers} outliers outside [{lower_bound:.2f}, {upper_bound:.2f}]")

   # Cap/clip extreme values to the upper and lower IQR fences
   df_filtered[col] = df_filtered[col].clip(lower=lower_bound, upper=upper_bound)

# Derive Price_Per_SqFt (in INR) after outlier treatment
df_filtered['Price_Per_SqFt_INR'] = ((df_filtered['Sale_Price_Lakhs'] * 100000) /
df_filtered['Area_SqFt']).round(2)
```

```
--- Task 7: Identifying & Handling Outliers (IQR Method) ---
Area_SqFt: Detected 0 outliers outside [-112.25, 3781.75]
Sale_Price_Lakhs: Detected 0 outliers outside [-73.28, 366.16]
```

TASK 6: Aggregate Data for Summary Statistics

```
print("\n--- Task 6: Data Aggregation ---")

# 1. Aggregation by Neighborhood
neighborhood_summary = df_filtered.groupby('Neighborhood').agg(
   Total_Listings=('Property_ID', 'count'),
   Avg_Sale_Price_Lakhs=('Sale_Price_Lakhs', 'mean'),
   Median_Sale_Price_Lakhs=('Sale_Price_Lakhs', 'median'),
   Avg_Price_Per_SqFt_INR=('Price_Per_SqFt_INR', 'mean'),
   Avg_Livability=('Livability_Index', 'first')
).round(2).reset_index().sort_values(by='Avg_Sale_Price_Lakhs', ascending=False)

print("\n1. Summary Statistics by Neighborhood:")
print(neighborhood_summary.to_string(index=False))

# 2. Pivot Table: Average Sale Price (Lakhs) by Neighborhood and Property Type
price_pivot = df_filtered.pivot_table(    values='Sale_Price_Lakhs',
index='Neighborhood',    columns='Property_Type',    aggfunc='mean'
).round(2)
```

```
print("\n2. Average Sale Price (₹ Lakhs) Pivot Table (Neighborhood vs. Property Type):")
print(price_pivot)
```

```
--- Task 6: Data Aggregation ---

1. Summary Statistics by Neighborhood:
 Neighborhood  Total_Listings  Avg_Sale_Price_Lakhs  Median_Sale_Price_Lakhs
Avg_Price_Per_SqFt_INR  Avg_Livability
        Baner              16                161.70                   159.12
11096.47             8.6
    Hinjewadi              12                154.43                   144.76
10714.23             7.9
        Wakad              11                153.66                   163.02
7035.90              8.1
      Kothrud              13                151.60                   168.23
9074.79              8.7
Koregaon Park              14                148.95                   148.50
11086.42             9.2
  Viman Nagar              17                147.29                   143.45
7665.64              8.9



2. Average Sale Price (₹ Lakhs) Pivot Table (Neighborhood vs. Property Type):
Property_Type  Apartment  Row House   Villa
Neighborhood
Baner             159.93     206.92  143.51
Hinjewadi         141.13     261.40  180.43
Koregaon Park     154.51     160.84   51.37
Kothrud           158.30      87.90  176.22
Viman Nagar       116.87     194.02  221.16
Wakad             160.70     198.10   84.62
```

TASK 5: Handle Categorical Variables (Encoding)

```
print("\n--- Task 5: Encoding Categorical Variables ---") df_encoded
= df_filtered.copy()

# 1. Ordinal / Label Encoding for ordered categories ('Furnishing_Status') furnishing_order
= {'Unfurnished': 0, 'Semi-Furnished': 1, 'Fully-Furnished': 2}
df_encoded['Furnishing_Encoded'] = df_encoded['Furnishing_Status'].map(furnishing_order)

# Label Encode 'Property_Type' for tree-based models le
= LabelEncoder()
df_encoded['Property_Type_Label'] = le.fit_transform(df_encoded['Property_Type'])

# 2. One-Hot Encoding for nominal categories ('Neighborhood' and 'Property_Type')
df_encoded = pd.get_dummies(
  df_encoded,
```

```
   columns=['Neighborhood', 'Property_Type'],
prefix=['Loc', 'Type'],    drop_first=True,
dtype=int
)

print("Encoded DataFrame Columns:") print(df_encoded.columns.tolist())

# Export final wrangled dataset
df_encoded.to_csv("Wrangled_RealEstate_Prices.csv", index=False) print("\nFinal
wrangled dataset saved to 'Wrangled_RealEstate_Prices.csv'.")
```

```
--- Task 5: Encoding Categorical Variables --- Encoded
DataFrame Columns:
['Property_ID', 'Area_SqFt', 'BHK', 'Furnishing_Status', 'Property_Age_Yrs',
'Sale_Date',
'Sale_Price_Lakhs', 'Metro_Distance_km', 'Livability_Index', 'Nearby_Schools_Count',
'Price_Per_SqFt_INR', 'Furnishing_Encoded', 'Property_Type_Label', 'Loc_Hinjewadi',
'Loc_Koregaon Park', 'Loc_Kothrud', 'Loc_Viman Nagar', 'Loc_Wakad', 'Type_Row House',
'Type_Villa']

Final wrangled dataset saved to 'Wrangled_RealEstate_Prices.csv'.
```

- **College Name : Pune Vidyarthi Griha's COLLEGE OF ENGINEERING, TECHNOLOGY AND MANAGEMENT,**
- **Department Name: Department of Artificial Intelligence and Data Science**
- **Student Name & Roll No.**
- **Class & Batch: BE**

EXPERIMENT 5: Data Visualization using matplotlib

Problem Statement: Analyzing Air Quality Index (AQI) Trends in a City

Prerequisites

pip install pandas numpy matplotlib

```
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages
(3.10.0)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.13/distpackages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2026.4)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/distpackages
(from matplotlib) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages
(from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.13/distpackages (from matplotlib) (4.66.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.13/distpackages (from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages
(from matplotlib) (26.3)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/distpackages
(from matplotlib) (3.3.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from
python-dateutil>=2.8.2->pandas) (1.17.0) #Step 0: Generate Sample City_Air_Quality.csv
```

```
import pandas as pd
import numpy as np
import os
```

```
if not os.path.exists("City_Air_Quality.csv"):
np.random.seed(42)
   dates = pd.date_range(start="2026-05-01", periods=180, freq="D")
n = len(dates)

   # Simulate seasonal curve (cleaner air during Monsoon July-Aug, higher pollution in May & Oct)
seasonal_factor = np.cos(np.linspace(0, 2 * np.pi, n))

   pm25 = np.clip(65 + 35 * seasonal_factor + np.random.normal(0, 15, n), 15, 250).round(2)
pm10 = np.clip(pm25 * 1.65 + np.random.normal(15, 20, n), 30, 420).round(2)     no2 =
np.clip(32 + 12 * seasonal_factor + np.random.normal(0, 6, n), 8, 95).round(2)     co =
np.clip(1.1 + 0.45 * seasonal_factor + np.random.normal(0, 0.2, n), 0.2, 3.5).round(2)

   # Calculate composite AQI and assign CPCB AQI Category
   aqi = np.clip(pm25 * 1.45 + pm10 * 0.35 + no2 * 0.25 + np.random.normal(0, 8, n), 35, 450).round(0)

   def get_aqi_bucket(val):
      if val <= 50: return "Good"       elif
val <= 100: return "Satisfactory"       elif
val <= 200: return "Moderate"       elif
val <= 300: return "Poor"       else:
return "Very Poor"

   aqi_category = [get_aqi_bucket(val) for val in aqi]
   dominant_pollutant = np.where(pm25 > 90, "PM2.5", np.where(pm10 > 150, "PM10", "NO2"))

   df_sample = pd.DataFrame({
      "Date": dates.strftime("%Y-%m-%d"),
      "Time": np.random.choice(["08:00:00", "14:00:00", "20:00:00"], n),
      "City": "Pune",
      "PM2.5": pm25,
      "PM10": pm10,
      "NO2": no2,
      "CO": co,
      "AQI": aqi,
      "Dominant_Pollutant": dominant_pollutant,
      "AQI_Category": aqi_category
   })

   df_sample.to_csv("City_Air_Quality.csv",                index=False)
print("Sample  'City_Air_Quality.csv'  generated  successfully.\n")
Sample 'City_Air_Quality.csv' generated successfully.

import  pandas  as  pd  import
numpy      as      np      import
matplotlib.pyplot  as  plt  import
matplotlib.dates as mdates
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.size'] = 10 plt.rcParams['axes.titlesize'] = 12
plt.rcParams['axes.titleweight'] = 'bold'
```

TASK 1 & 2: Import and Explore the Dataset

```
df = pd.read_csv("City_Air_Quality.csv")
print("--- Task 1 & 2: Dataset Structure and Summary ---")
print(f"Dataset Shape: {df.shape}") print("\nData Types &
Non-Null Counts:")
print(df.info())
print("\nDescriptive Statistics:")
print(df.describe().round(2))
print("\nFirst 5 Rows:")
print(df.head())
```

```
--- Task 1 & 2: Dataset Structure and Summary ---
Dataset Shape: (180, 10)

Data Types & Non-Null Counts:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 180 entries, 0 to 179 Data
columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
0   Date                180 non-null    object
1   Time                180 non-null    object
2   City                180 non-null    object
3   PM2.5               180 non-null    float64
4   PM10                180 non-null    float64
5   NO2                 180 non-null    float64
6   CO                  180 non-null    float64
7   AQI                 180 non-null    float64
8   Dominant_Pollutant  180 non-null    object  9   AQI_Category        180 non-null
    object  dtypes: float64(5), object(5) memory usage: 14.2+ KB
None

Descriptive Statistics:
        PM2.5    PM10     NO2      CO     AQI
count  180.00  180.00  180.00  180.00  180.00
mean    65.12  123.11   31.78    1.10  146.62 std
28.48   49.88   10.55    0.38   59.79 min
15.00   30.00    8.01    0.20   35.00 25%
40.79   82.14   23.27    0.83   93.75
50%     63.34  125.40   32.57    1.10  146.50 75%
87.90  161.10   39.90    1.41  192.50 max
140.80  230.78   55.50    1.86  289.00

First 5 Rows:
         Date      Time  City   PM2.5    PM10    NO2    CO    AQI  \
0  2026-05-01  20:00:00  Pune  107.45  204.81  47.12  1.72  242.0
1  2026-05-02  14:00:00  Pune   97.90  159.39  53.19  1.32  197.0
2  2026-05-03  14:00:00  Pune  109.63  174.47  43.32  1.65  220.0
```

```
3  2026-05-04  14:00:00  Pune  122.65  227.02  46.34  1.84  275.0    4  2026-05-05
   08:00:00  Pune   96.14  169.16  48.02  1.05  212.0
```

```
  Dominant_Pollutant AQI_Category
0              PM2.5         Poor
1              PM2.5     Moderate
2              PM2.5         Poor
3              PM2.5         Poor
4              PM2.5         Poor
```

TASK 3: Identify & Prepare Relevant Variables

```
# Convert Date to datetime object and sort chronologically
df['Date'] = pd.to_datetime(df['Date'])
df = df.sort_values('Date').reset_index(drop=True)

# Extract Month for periodic bar chart comparisons
df['Month'] = df['Date'].dt.to_period('M').astype(str)
df['Month_Name'] = df['Date'].dt.strftime('%b %Y')

# Clean any potential missing numerical values using linear interpolation pollutant_cols
= ['PM2.5', 'PM10', 'NO2', 'CO', 'AQI']
df[pollutant_cols] = df[pollutant_cols].interpolate(method='linear')

# Calculate 7-Day Rolling Average for smoother trend visualization df['AQI_7Day_MA']
= df['AQI'].rolling(window=7, min_periods=1).mean()
```

TASK 4 & 9: Line Plot of Overall AQI Trend Over Time fig,

```
ax = plt.subplots(figsize=(13, 5))

ax.plot(df['Date'], df['AQI'], color='#adb5bd', alpha=0.6, linewidth=1.2, label='Daily AQI')
ax.plot(df['Date'], df['AQI_7Day_MA'], color='#d90429', linewidth=2.5, label='7-Day Moving Average')

# Add horizontal reference lines for AQI safety thresholds ax.axhline(100, color='#2b9348',
linestyle='--', linewidth=1.2, label='Satisfactory Limit (100)') ax.axhline(200, color='#f77f00',
linestyle='--', linewidth=1.2, label='Moderate Limit (200)')

ax.set_title('Task 4: Overall Air Quality Index (AQI) Trend Over Time')
ax.set_xlabel('Date') ax.set_ylabel('AQI Value')
ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
ax.legend(loc='upper right', frameon=True) ax.grid(True,
linestyle=':', alpha=0.6)

plt.tight_layout() plt.show()
```



TASK 5 & 9: Separate Line Plots for Individual Pollutants

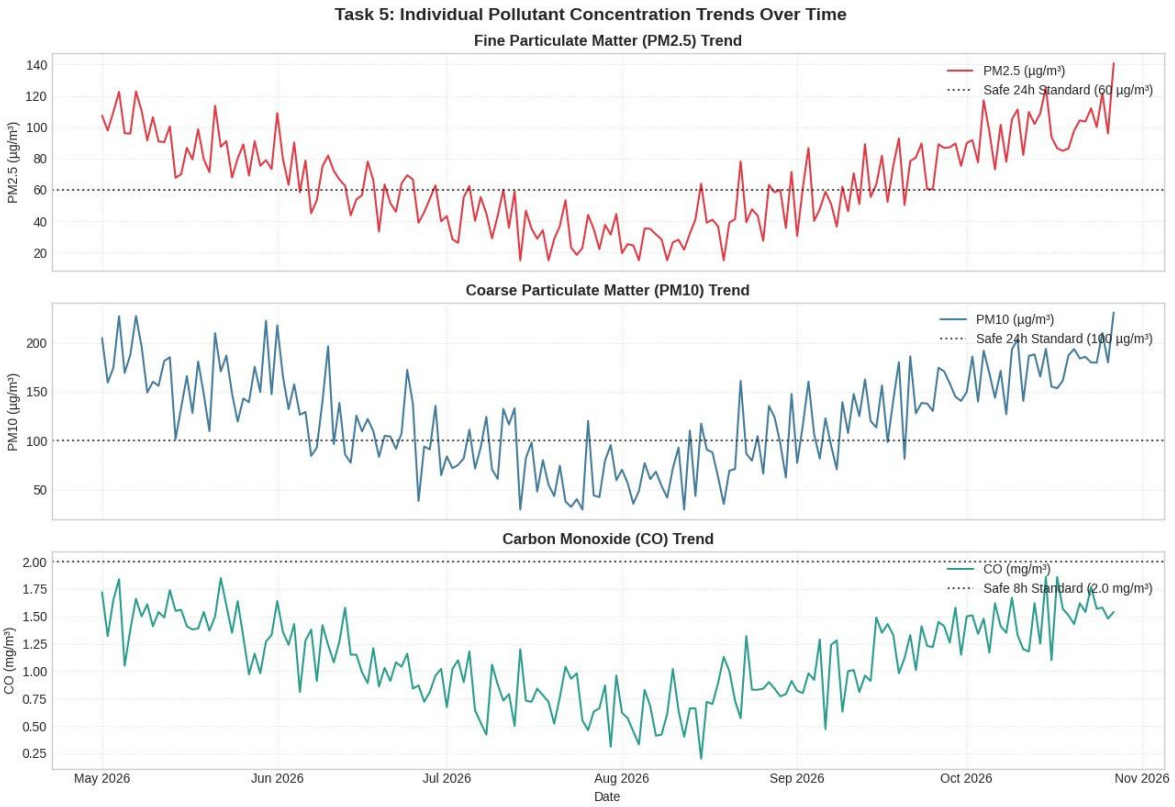
```
fig, axes = plt.subplots(3, 1, figsize=(13, 9), sharex=True)
fig.suptitle('Task 5: Individual Pollutant Concentration Trends Over Time', fontsize=14, fontweight='bold')

# 1. PM2.5 Trend axes[0].plot(df['Date'], df['PM2.5'], color='#e63946', linewidth=1.5,
label='PM2.5 (µg/m³)') axes[0].axhline(60, color='black', linestyle=':', alpha=0.7, label='Safe
24h Standard (60 µg/m³)') axes[0].set_ylabel('PM2.5 (µg/m³)')
axes[0].set_title('Fine Particulate Matter (PM2.5) Trend')
axes[0].legend(loc='upper right') axes[0].grid(True,
linestyle=':', alpha=0.6)

# 2. PM10 Trend
axes[1].plot(df['Date'], df['PM10'], color='#457b9d', linewidth=1.5, label='PM10 (µg/m³)')
axes[1].axhline(100, color='black', linestyle=':', alpha=0.7, label='Safe 24h Standard (100 µg/m³)')
axes[1].set_ylabel('PM10 (µg/m³)')
axes[1].set_title('Coarse Particulate Matter (PM10) Trend')
axes[1].legend(loc='upper right') axes[1].grid(True,
linestyle=':', alpha=0.6)

# 3. Carbon Monoxide (CO) Trend
axes[2].plot(df['Date'], df['CO'], color='#2a9d8f', linewidth=1.5, label='CO (mg/m³)')
axes[2].axhline(2.0, color='black', linestyle=':', alpha=0.7, label='Safe 8h Standard (2.0 mg/m³)')
axes[2].set_ylabel('CO (mg/m³)') axes[2].set_xlabel('Date')
axes[2].set_title('Carbon Monoxide (CO) Trend')
axes[2].xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
axes[2].legend(loc='upper right') axes[2].grid(True, linestyle=':',
alpha=0.6)

plt.tight_layout() plt.show()
```



TASK 6 & 9: Bar Plot & Stacked Bar Plot Across Time Periods

```
# Aggregate monthly average AQI and pollutant levels
monthly_Order = df.groupby('Month', sort=True).agg(
   Month_Label=('Month_Name', 'first'),
   Avg_AQI=('AQI', 'mean'),
   Avg_PM25=('PM2.5', 'mean'),
   Avg_PM10=('PM10', 'mean'),
   Avg_NO2=('NO2', 'mean')
).reset_index()

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5.5))

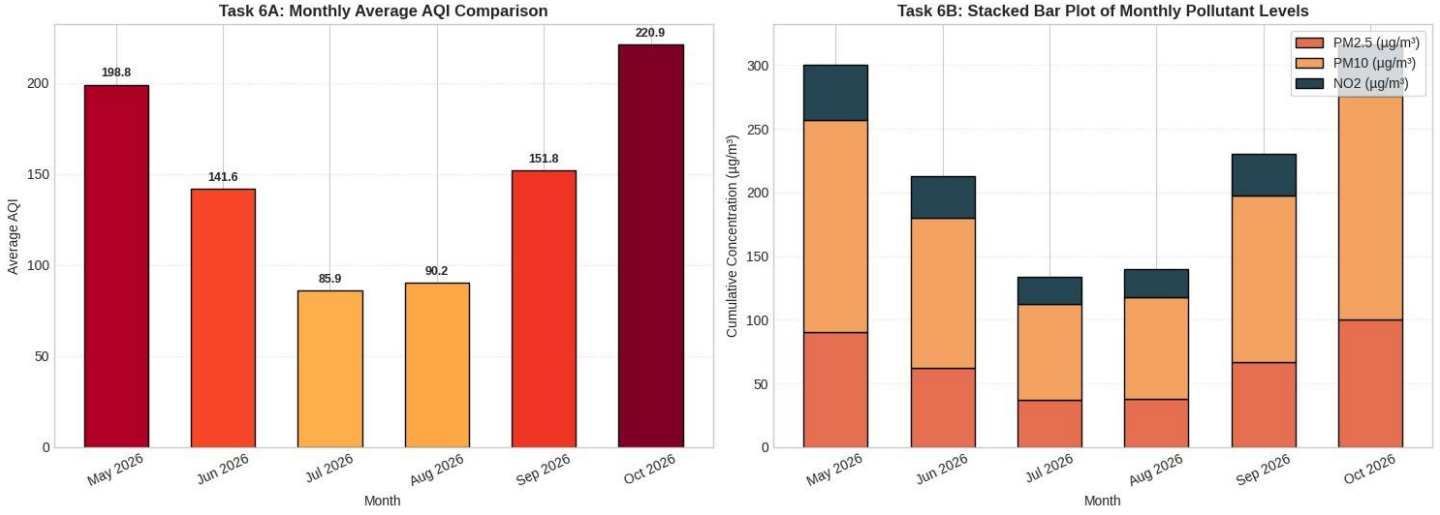
# 6A. Standard Bar Plot: Monthly Average AQI
bars = ax1.bar(
monthly_Order['Month_Label'],
monthly_Order['Avg_AQI'],
   color=plt.cm.YlOrRd(monthly_Order['Avg_AQI'] / monthly_Order['Avg_AQI'].max()),
edgecolor='black',    width=0.6
)
ax1.set_title('Task 6A: Monthly Average AQI Comparison')
ax1.set_xlabel('Month') ax1.set_ylabel('Average AQI')
ax1.tick_params(axis='x', rotation=25) ax1.grid(axis='y',
linestyle=':', alpha=0.6)

# Annotate bar values on top
for bar in bars:      height =
bar.get_height()
   ax1.annotate(f'{height:.1f}', xy=(bar.get_x() + bar.get_width() / 2, height),
            xytext=(0, 4), textcoords="offset points", ha='center', va='bottom', fontsize=9, fontweight='bold')
```

```
# 6B. Stacked Bar Plot: Monthly Pollutant Composition x_labels
= monthly_Order['Month_Label']
p1 = ax2.bar(x_labels, monthly_Order['Avg_PM25'], label='PM2.5 (µg/m³)', color='#e76f51',
edgecolor='black', width=0.6)
p2 = ax2.bar(x_labels, monthly_Order['Avg_PM10'], bottom=monthly_Order['Avg_PM25'], label='PM10
(µg/m³)', color='#f4a261', edgecolor='black', width=0.6)
p3 = ax2.bar(x_labels, monthly_Order['Avg_NO2'], bottom=monthly_Order['Avg_PM25'] +
monthly_Order['Avg_PM10'], label='NO2 (µg/m³)', color='#264653', edgecolor='black', width=0.6)

ax2.set_title('Task 6B: Stacked Bar Plot of Monthly Pollutant Levels') ax2.set_xlabel('Month')
ax2.set_ylabel('Cumulative Concentration (µg/m³)')
ax2.tick_params(axis='x', rotation=25) ax2.legend(loc='upper
right', frameon=True) ax2.grid(axis='y', linestyle=':',
alpha=0.6)

plt.tight_layout() plt.show()
```



TASK 7 & 9: Box Plot & Violin Plot for AQI Distributions

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5.5))

# 7A. Box Plot of AQI grouped by Dominant Pollutant Category
pollutant_groups = ['PM2.5', 'PM10', 'NO2']
box_data = [df[df['Dominant_Pollutant'] == grp]['AQI'].dropna().values for grp in pollutant_groups if grp
in df['Dominant_Pollutant'].values]
valid_labels = [grp for grp in pollutant_groups if grp in df['Dominant_Pollutant'].values]

bp = ax1.boxplot(    box_data,
tick_labels=valid_labels,    patch_artist=True,
notch=False,    boxprops=dict( edgecolor='black',
linewidth=1.2),    medianprops=dict(color='yellow',
linewidth=2)
)
```

```
box_colors = ['#ef476f', '#118ab2', '#06d6a0'] for patch, color in
zip(bp['boxes'], box_colors[:len(valid_labels)]):
patch.set_facecolor(color)    patch.set_alpha(0.8)

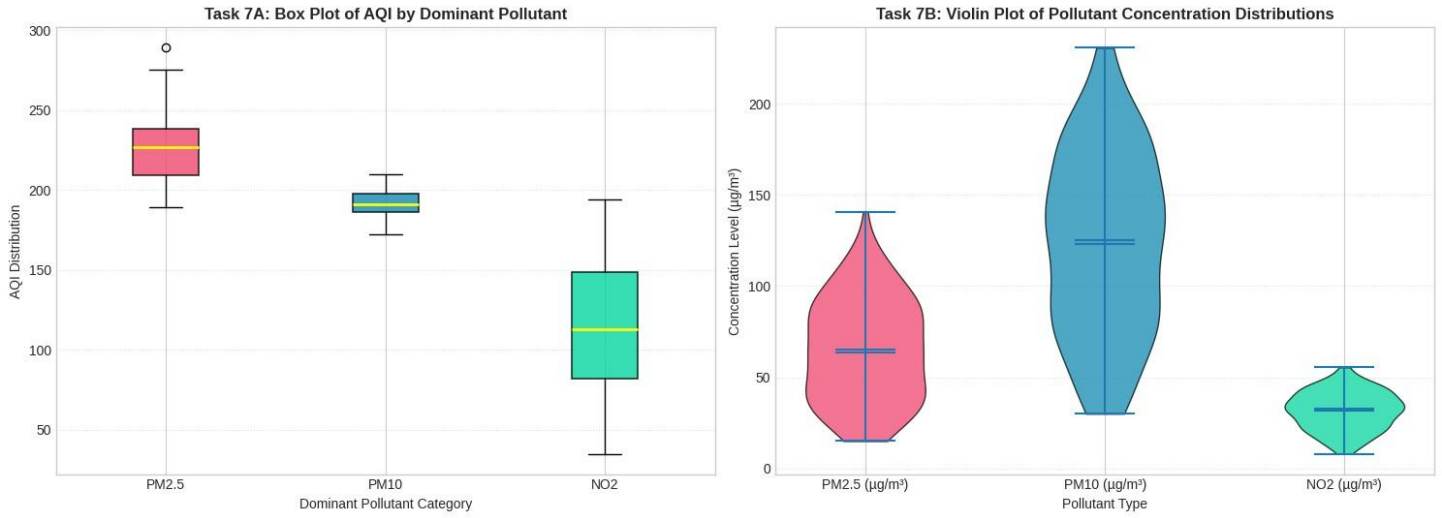
ax1.set_title('Task 7A: Box Plot of AQI by Dominant Pollutant')
ax1.set_xlabel('Dominant Pollutant Category')
ax1.set_ylabel('AQI Distribution') ax1.grid(axis='y',
linestyle=':', alpha=0.6)

# 7B. Violin Plot of Individual Pollutant Concentrations
violin_cols = ['PM2.5', 'PM10', 'NO2']
violin_data = [df[col].dropna().values for col in violin_cols]

vp = ax2.violinplot(violin_data, showmeans=True, showmedians=True) for
i, body in enumerate(vp['bodies']):
  body.set_facecolor(box_colors[i])
body.set_edgecolor('black')    body.set_alpha(0.75)

ax2.set_xticks([1, 2, 3])
ax2.set_xticklabels(['PM2.5 (µg/m³)', 'PM10 (µg/m³)', 'NO2 (µg/m³)'])
ax2.set_title('Task 7B: Violin Plot of Pollutant Concentration Distributions')
ax2.set_xlabel('Pollutant Type') ax2.set_ylabel('Concentration Level
(µg/m³)') ax2.grid(axis='y', linestyle=':', alpha=0.6)

plt.tight_layout() plt.show()
```



TASK 8 & 9: Scatter Plot & Bubble Chart (AQI vs Pollutants)

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

# 8A. Scatter Plot: PM2.5 vs AQI with Linear Trendline
sc1 = ax1.scatter( df['PM2.5'], df['AQI'],
c=df['AQI'], cmap='plasma', alpha=0.8,
edgecolors='k', linewidth=0.5,
  s=50
)
```

```
# Fit and plot linear regression line m, b =
np.polyfit(df['PM2.5'], df['AQI'], 1)
ax1.plot(df['PM2.5'], m * df['PM2.5'] + b, color='red', linestyle='--', linewidth=2, label=f'Trendline
(Slope={m:.2f})')

cbar1 = plt.colorbar(sc1, ax=ax1)
cbar1.set_label('AQI Intensity') ax1.set_title('Task
8A: Scatter Plot of PM2.5 vs. AQI')
ax1.set_xlabel('PM2.5 Concentration (µg/m³)')
ax1.set_ylabel('Air Quality Index (AQI)')
ax1.legend(loc='upper left')
ax1.grid(True, linestyle=':', alpha=0.6)

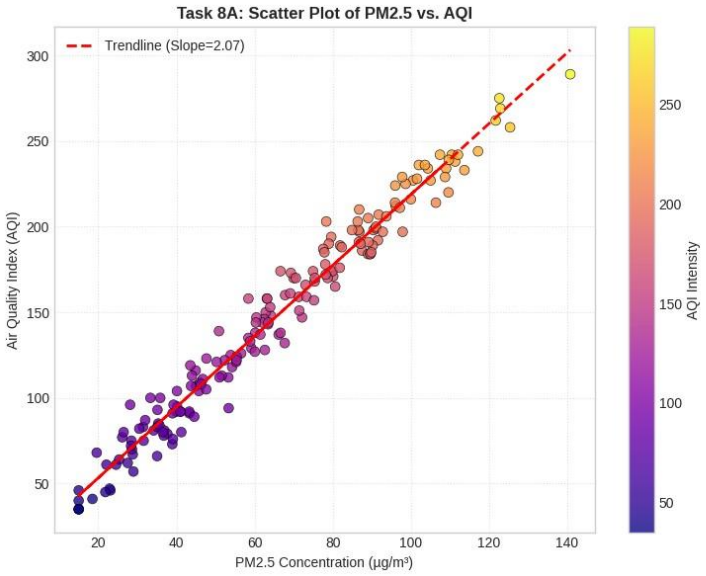
# 8B. Bubble Chart: PM10 vs AQI (Bubble Size = CO Level, Color = PM2.5)
bubble_sizes = df['CO'] * 75  # Scale CO values so bubble radii are visually distinct
sc2 = ax2.scatter(    df['PM10'],    df['AQI'],
   s=bubble_sizes,
c=df['PM2.5'],
cmap='viridis',    alpha=0.75,
edgecolors='black',
linewidth=0.8
)

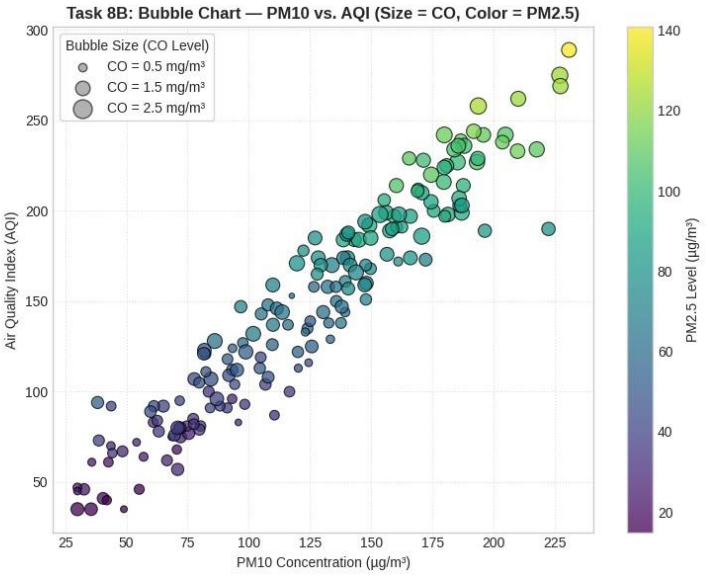

cbar2 = plt.colorbar(sc2, ax=ax2) cbar2.set_label('PM2.5
Level (µg/m³)')

# Create dummy scatter points for bubble size legend for
co_val in [0.5, 1.5, 2.5]:
   ax2.scatter([], [], c='gray', alpha=0.6, s=co_val * 75, edgecolors='black', label=f'CO = {co_val} mg/m³')

ax2.set_title('Task 8B: Bubble Chart — PM10 vs. AQI (Size = CO, Color = PM2.5)')
ax2.set_xlabel('PM10 Concentration (µg/m³)') ax2.set_ylabel('Air Quality Index
(AQI)')
ax2.legend(scatterpoints=1, frameon=True, title='Bubble Size (CO Level)', loc='upper left') ax2.grid(True,
linestyle=':', alpha=0.6)

plt.tight_layout() plt.show()
```


Task 8A: Scatter Plot of PM2.5 vs. AQI
Trendline (Slope=2.07)
Air Quality Index (AQI)
PM2.5 Concentration (µg/m³)
AQI Intensity


Task 8B: Bubble Chart — PM10 vs. AQI (Size = CO, Color = PM2.5)
Bubble Size (CO Level)
CO = 0.5 mg/m³
CO = 1.5 mg/m³
CO = 2.5 mg/m³
Air Quality Index (AQI)
PM10 Concentration (µg/m³)
PM2.5 Level (µg/m³)

- **College Name : Pune Vidyarthi Griha's COLLEGE OF ENGINEERING, TECHNOLOGY AND MANAGEMENT,**
- **Department Name: Department of Artificial Intelligence and Data Science**
- **Student Name & Roll No.**
- **Class & Batch: BE**

EXPERIMENT 6: Data Aggregation

Problem Statement: Analyzing Sales Performance by Region in a Retail Company

Prerequisites

pip install pandas numpy matplotlib seaborn

```
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages
(3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.13/dist-packages (0.13.2)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.13/distpackages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2026.4)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/distpackages
(from matplotlib) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages
(from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/distpackages
(from matplotlib) (4.66.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/distpackages
(from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages
(from matplotlib) (26.3)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/distpackages
(from matplotlib) (3.3.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from
python-dateutil>=2.8.2->pandas) (1.17.0)
```

Step 0: Generate Sample Retail_Sales_Data.csv

```
import pandas as pd
import numpy as np
import os
```

```
if not os.path.exists("Retail_Sales_Data.csv"):
    np.random.seed(42)
n = 300

    regions = ['West', 'South', 'North', 'East', 'Central']
    categories = ['Electronics', 'Apparel', 'Home & Kitchen', 'Groceries', 'Beauty & Personal Care']
payment_modes = ['UPI', 'Credit Card', 'Debit Card', 'Net Banking', 'Cash']

    # Assign regional weights so top-performing regions emerge clearly     region_choices
= np.random.choice(regions, n, p=[0.30, 0.25, 0.20, 0.15, 0.10])     category_choices =
np.random.choice(categories,  n,  p=[0.25,  0.25,  0.20,  0.20,  0.10])         quantities  =
np.random.randint(1, 8, n)

    # Set realistic unit price ranges per category (in INR)
base_prices = {
        'Electronics': (8000, 45000),
        'Apparel': (800, 4500),
        'Home & Kitchen': (1200, 12000),
        'Groceries': (250, 2500),
        'Beauty & Personal Care': (300, 3000)
    }

    unit_prices = [
        np.round(np.random.uniform(*base_prices[cat]), 2)
for cat in category_choices
    ]

    sales_amounts = np.round(quantities * np.array(unit_prices), 2)

    df_sample = pd.DataFrame({
        'Transaction_ID': [f'TXN-{10000+i}' for i in range(n)],
        'Transaction_Date': pd.date_range(start='2026-01-01', periods=n, freq='18h').strftime('%Y-%m-
%d'),
        'Region': region_choices,
        'Product_Category': category_choices,
        'Quantity_Sold': quantities,
        'Unit_Price': unit_prices,
        'Sales_Amount': sales_amounts,
        'Payment_Mode': np.random.choice(payment_modes, n, p=[0.45, 0.25, 0.15, 0.10, 0.05])
    })

    # Inject a few missing values to demonstrate safe pre-aggregation checks
df_sample.loc[10:12, 'Sales_Amount'] = np.nan

    df_sample.to_csv("Retail_Sales_Data.csv",                 index=False)
print("Sample  'Retail_Sales_Data.csv'  generated  successfully.\n")
Sample 'Retail_Sales_Data.csv' generated successfully.
```

```
import pandas as pd import
numpy as np import
matplotlib.pyplot as plt import
seaborn as sns




# Set visual theme sns.set_theme(style="whitegrid")
plt.rcParams['font.size'] = 10
plt.rcParams['axes.titlesize'] = 12
plt.rcParams['axes.titleweight'] = 'bold'
```

TASK 1 & 2: Import and Explore the Dataset

```
df = pd.read_csv("Retail_Sales_Data.csv")

print("--- Task 1 & 2: Dataset Structure & Exploration ---") print(f"Dataset
Shape: {df.shape}")
print("\nColumn Data Types & Non-Null Counts:")
print(df.info()) print("\nSummary Statistics:")
print(df.describe().round(2)) print("\nFirst 5
Records:") print(df.head())
--- Task 1 & 2: Dataset Structure & Exploration ---
Dataset Shape: (300, 8)

Column Data Types & Non-Null Counts:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 300 entries, 0 to 299 Data
columns (total 8 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
0   Transaction_ID    300 non-null    object
1   Transaction_Date  300 non-null    object
2   Region            300 non-null    object
3   Product_Category  300 non-null    object
4   Quantity_Sold     300 non-null    int64
5   Unit_Price        300 non-null    float64
6   Sales_Amount      297 non-null    float64  7   Payment_Mode      300 non-null    object
    dtypes: float64(2), int64(1), object(5)
memory usage: 18.9+ KB
None

Summary Statistics:
       Quantity_Sold  Unit_Price  Sales_Amount
count         300.00      300.00        297.00
mean            4.09     9348.21      38362.51
std             2.06    12439.86      61097.36
min             1.00      279.25        349.65
25%             2.00     1647.57       5665.26
```

```
50%             4.00     3001.03      11569.80
75%             6.00    10956.04      42937.86
max             7.00    44923.56     307504.12

First 5 Records:
  Transaction_ID Transaction_Date   Region Product_Category  Quantity_Sold  \
0      TXN-10000       2026-01-01    South      Electronics              5
1      TXN-10001       2026-01-01  Central   Home & Kitchen              2
2      TXN-10002       2026-01-02    North   Home & Kitchen              4
3      TXN-10003       2026-01-03    North   Home & Kitchen              2    4      TXN-
       10004        2026-01-04     West         Groceries               5       Unit_Price
       Sales_Amount Payment_Mode
0     40440.33     202201.65          UPI
1     4894.48       9788.96          UPI
2     10069.58      40278.32          UPI
3     2394.82       4789.64          UPI
4     2154.52      10772.60          UPI
```

TASK 3: Identify & Clean Relevant Variables

```
relevant_cols = ['Region', 'Product_Category', 'Quantity_Sold', 'Sales_Amount'] print("\n---
Task 3: Checking Relevant Variables ---")
print("Missing values in key columns:\n", df[relevant_cols].isnull().sum())

# Recompute or impute any missing Sales_Amount values before aggregation if
df['Sales_Amount'].isnull().sum() > 0:    if 'Unit_Price' in df.columns:
df['Sales_Amount'] = df['Sales_Amount'].fillna(df['Quantity_Sold'] * df['Unit_Price'])
else:
    df['Sales_Amount'] = df['Sales_Amount'].fillna(df['Sales_Amount'].median())

# Standardize string formatting in grouping columns df['Region']
= df['Region'].astype(str).str.strip().str.title()
df['Product_Category'] = df['Product_Category'].astype(str).str.strip().str.title()
```

```
--- Task 3: Checking Relevant Variables --- Missing
values in key columns:
 Region               0 Product_Category
0
Quantity_Sold        0
Sales_Amount         3 dtype:
int64
```

TASK 4: Group by Region & Calculate Total Sales

```
regional_sales = df.groupby('Region', as_index=False).agg(
  Total_Transactions=('Transaction_ID', 'count'),
  Total_Units_Sold=('Quantity_Sold', 'sum'),
  Total_Sales_Amount=('Sales_Amount', 'sum'),
  Average_Order_Value=('Sales_Amount', 'mean') ).round(2)

# Sort descending by Total_Sales_Amount
```

```
regional_sales = regional_sales.sort_values(by='Total_Sales_Amount',
ascending=False).reset_index(drop=True)

# Add percentage share of overall revenue
total_company_sales = regional_sales['Total_Sales_Amount'].sum()
regional_sales['Revenue_Share_Pct'] = ((regional_sales['Total_Sales_Amount'] / total_company_sales) *
100).round(2)

print("\n--- Task 4: Sales Aggregated by Region ---") print(regional_sales.to_string(index=False))
```

```
--- Task 4: Sales Aggregated by Region ---
 Region   Total_Transactions   Total_Units_Sold   Total_Sales_Amount   Average_Order_Value
Revenue_Share_Pct
   West                  95               406          3933559.22              41405.89
33.91
  South                  70               277          2469182.68              35274.04
21.29
  North                  59               229          1892976.70              32084.35
16.32
   East                  47               202          1733804.53              36889.46
14.95
Central                  29               113           1569625.60               54125.02
13.53
```

TASK 6: Identify Top-Performing Regions

```
top_region = regional_sales.iloc[0] top_3_regions
= regional_sales.head(3)

print("\n--- Task 6: Top-Performing Regions ---")
print(f"Top-Performing Region : {top_region['Region']} (₹{top_region['Total_Sales_Amount']:,.2f} |
{top_region['Revenue_Share_Pct']}% of total revenue)") print("\nTop
3 Regions by Sales Amount:")
print(top_3_regions[['Region', 'Total_Sales_Amount', 'Revenue_Share_Pct']].to_string(index=False))
```

```
--- Task 6: Top-Performing Regions ---
Top-Performing Region : West (₹3,933,559.22 | 33.91% of total revenue)

Top 3 Regions by Sales Amount:
Region  Total_Sales_Amount  Revenue_Share_Pct
  West          3933559.22              33.91
 South          2469182.68              21.29
 North          1892976.70              16.32
```

TASK 5: Visualize Sales Distribution by Region (Bar & Pie Chart)

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5.5))
fig.suptitle('Task 5: Regional Sales Performance Distribution', fontsize=14, fontweight='bold')

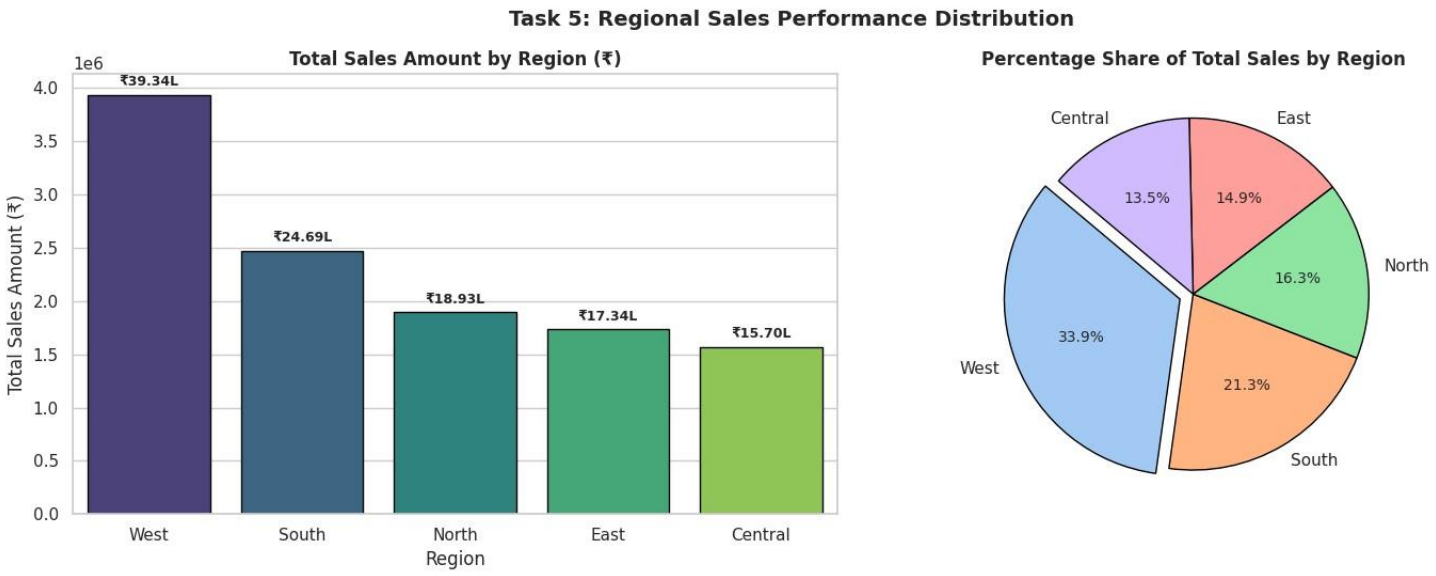
# 5A. Bar Plot of Total Sales by Region
bars = sns.barplot(
data=regional_sales,
  x='Region',
  y='Total_Sales_Amount',
  hue='Region',
palette='viridis',
edgecolor='black',
ax=ax1,   legend=False
)
ax1.set_title('Total Sales Amount by Region (₹)') ax1.set_xlabel('Region')
ax1.set_ylabel('Total Sales Amount (₹)')



# Add value labels on top of each bar for
bar in ax1.patches:
  height = bar.get_height()
ax1.annotate(
f'₹{height/1e5:.2f}L',
     xy=(bar.get_x() + bar.get_width() / 2, height),
xytext=(0, 4),
     textcoords="offset points",
     ha='center', va='bottom', fontsize=9, fontweight='bold'
  )

# 5B. Pie Chart of Regional Revenue Share
explode_vals = [0.08 if i == 0 else 0 for i in range(len(regional_sales))]  # Highlight #1 region
ax2.pie(   regional_sales['Total_Sales_Amount'],   labels=regional_sales['Region'],
  autopct='%1.1f%%',
startangle=140,
explode=explode_vals,
  colors=sns.color_palette('pastel'),
  wedgeprops={'edgecolor': 'black', 'linewidth': 1}
)
ax2.set_title('Percentage Share of Total Sales by Region')

plt.tight_layout() plt.show()
```



TASK 7: Group by Region and Product Category

```
region_category_sales = df.groupby(['Region', 'Product_Category'], as_index=False).agg(
   Total_Units_Sold=('Quantity_Sold', 'sum'),
   Total_Sales_Amount=('Sales_Amount', 'sum') ).round(2)

print("\n--- Task 7A: Sales Aggregated by Region & Product Category (Long Format) ---")
print(region_category_sales.head(10).to_string(index=False))



# Create a Pivot Table (Cross-Tabulation) for easy multi-dimensional comparison & stacked plotting
sales_pivot = df.pivot_table(    index='Region',    columns='Product_Category',
values='Sales_Amount',    aggfunc='sum',
   fill_value=0
).round(2)

# Reorder rows to match the top-performing region order
sales_pivot = sales_pivot.loc[regional_sales['Region']]

print("\n--- Task 7B: Region vs. Product Category Pivot Table (₹) ---")
print(sales_pivot)
```

```
--- Task 7A: Sales Aggregated by Region & Product Category (Long Format) ---
 Region       Product_Category  Total_Units_Sold  Total_Sales_Amount
Central                Apparel                23            75270.28
Central Beauty & Personal Care                11            11570.65
Central            Electronics                48          1285512.26
Central              Groceries                 3             7152.06
Central         Home & Kitchen                28           190120.35
   East                Apparel                53           124750.32
   East Beauty & Personal Care                 9            13479.79
   East            Electronics                47          1372327.27
```

```
   East                Groceries                   56             53568.80
East         Home & Kitchen                 37           169678.35

--- Task 7B: Region vs. Product Category Pivot Table (₹) ---
Product_Category    Apparel  Beauty & Personal Care  Electronics  Groceries  \
Region
West              185419.09               103905.72   3040866.68  120638.42
South             160066.92                64396.48   1853250.30   80358.53
North             160539.70                71742.37   1279002.57   47223.10
East              124750.32                13479.79   1372327.27   53568.80    Central
75270.28               11570.65   1285512.26    7152.06

Product_Category  Home & Kitchen
Region
West                   482729.31
South                  311110.45
North                  334468.96
East                   169678.35
Central                190120.35
```

TASK 8: Grouped Bar Plot & Stacked Bar Plot (Region vs. Category)

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('Task 8: Sales Comparison Across Regions and Product Categories', fontsize=14, fontweight='bold')




# 8A. Grouped Bar Plot using Seaborn
sns.barplot(
    data=region_category_sales,
x='Region',    y='Total_Sales_Amount',
hue='Product_Category',
order=regional_sales['Region'],
    palette='Set2',
edgecolor='black',    ax=ax1
)
ax1.set_title('Task 8A: Grouped Bar Plot (Region × Product Category)') ax1.set_xlabel('Region')
ax1.set_ylabel('Total Sales Amount (₹)')
ax1.legend(title='Product Category', loc='upper right')

# 8B. Stacked Bar Plot using Pandas/Matplotlib on the Pivot Table
sales_pivot.plot(    kind='bar',    stacked=True,
colormap='tab20c',    edgecolor='black',    width=0.65,    ax=ax2
)
ax2.set_title('Task 8B: Stacked Bar Plot of Category Composition per Region') ax2.set_xlabel('Region')
```

```
ax2.set_ylabel('Cumulative Sales Amount (₹)') ax2.tick_params(axis='x',
rotation=0)
ax2.legend(title='Product Category', loc='upper right')

plt.tight_layout() plt.show()
```

